Predicting Property Prices in a Specific Location Using Machine Learning

Capstone Project – 1 · Project Summary Report: data cleaning, exploratory analysis, encoding, scaling and PCA, model comparison and held-out evaluation

Prepared by Srishti Sharma · Capstone Project – 1

Dataset: Property_data (1) (1).csv · 1,460 property sales × 81 columns · Ames, Iowa, sales 2006–2010 · split 1,168 training / 292 held-out rows (seed 205)

#	CONTENTS	PAGE NO.
1.	Introduction to the Project	1
2.	Objectives of the Project	3
3.	Flow Chart of Operations	6
4.	Python Codes	7
5.	Screenshot of the Outputs	16
6.	Report on EDA	22
7.	Learning Outcomes	30
8.	Conclusion	30
9.	Citations	38
10.	Appendices A–D	39

1. Introduction to the Project

1.1 Background and problem statement

Pricing a property correctly matters to everyone in a sale. Sellers risk leaving money on the table or leaving the property unsold, buyers need to know whether an asking price is fair, and agents and lenders need a consistent, data-driven benchmark that complements human judgement. This project builds a supervised **regression** model that predicts a residential property's sale price (PropPrice, in dollars) from its physical, structural, quality and sale-related characteristics, and identifies which characteristics are most strongly associated with price.

The brief asks for a machine learning model for "a specific location". The supplied data describe house sales in **Ames, Iowa** between 2006 and 2010 (a renamed adaptation of the Ames Housing dataset of De Cock, 2011). The model therefore predicts prices in that market only; Section 8.5 states what that does and does not allow us to conclude.

1.2 The dataset

ITEM	DETAIL
FILE AND SIZE	One CSV, 1,460 rows × 81 columns: PropertyID, 79 predictors and the target PropPrice. Every PropertyID is unique and no row repeats another once the ID is ignored.
PREDICTOR TYPES	34 numeric and 45 text predictors. After the metadata review (Section 2.2): 34 numeric, 3 binary, 19 ordinal and 23 nominal.

TARGET	PropPrice, sale price in dollars. Training rows: mean $181,554, median $163,995, range $35,311 to $755,000, skewness +1.95.
PERIOD AND PLACE	Sales in Ames, Iowa, 2006–2010; houses built 1872–2010.
BRIEF VERSUS SUPPLIED FILES	The brief lists train.csv, test.csv, data_description.txt and sample_submission.csv. Only one property-data CSV was supplied, so a reproducible random 80/20 split was made (1,168 training and 292 held-out rows, seed 205, test-ID fingerprint 992f3853012a11f8).
COLUMNS CONTAINING THE TEXT NA	19 columns. In this file "NA" usually means "the feature does not exist" (no pool, no alley, no garage), not "unknown" (Section 6.2).
COLUMN RULE FROM THE BRIEF	Do not drop any column other than PropertyID. All 79 predictors enter the modelling pipeline.

1.3 Headline results

1460 property sales, 79 predictors	**1,168 / 292** training / held-out rows (80 / 20)	**0.847** cross-validated R^2 (± 0.082), training rows only	**0.930** held-out R^2 (95% CI 0.912–0.948)
$ 12,828 held-out mean absolute error	**$18,512** held-out RMSE (log RMSE 0.117)	**Lasso** final model, fitted to log price	**71 of 200** encoded terms non-zero in the final fit

- **Final model.** Lasso regression on log price, with learned preprocessing fitted within training folds. Training-only nested cross-validation did not establish a clear RMSE advantage among the shortlisted models. Lasso had lower MAE than Gradient Boosting in all five outer folds and was selected using the documented preference for simplicity and interpretability.
- **Accuracy against the brief.** The brief expects R^2 of 0.75–0.85, and above 0.85 is better. Training-only cross-validation gives R^2 0.847, inside the expected band and just under 0.85. On the 292 held-out sales the model reaches R^2 0.930, above 0.85.
- **Interpreting the two scores.** The held-out partition contains none of the four training properties above 4,000 sq ft. This is consistent with an easier test sample, but does not establish the sole cause of its stronger score. Training-only cross-validation is the more cautious internal reference, although repeated analytical choices on the same training dataset can also introduce optimism (Section 8.2).
- **What drives price.** Living area (GrLivArea) and overall quality (OverallQual) dominate. Shuffling them raises validation RMSE by $14,871 and $8,978; the next feature adds $2,356 or less. Neighborhood separates the market into clear tiers.
- **Where it is weak.** The most expensive quarter of sales (MAE $19,326) and new-construction "Partial" sales (MAE $19,743) are predicted worst, and the 80% prediction intervals cover only about 67% and 70% of those groups.

> **So what:** The model is a validated analytical prototype for this Ames data, not a production valuation tool. Use cross-validated R^2 0.847 and RMSE of about $30,700 as the expected accuracy, treat the held-out 0.930 as supporting evidence, and read Section 8.5 before applying it elsewhere.

1.4 Disclosure about the held-out rows

An earlier version of the notebook scored an untuned Gradient Boosting model on the same 292 held-out rows (R^2 0.912). That version was revised after training-only diagnostics exposed an extrapolation problem, and no choice was based on test results. The current run logs two test-set accesses: predictor-only split checks in Section 3.4 and one final evaluation in Section 11.1. Because of the earlier exposure the held-out rows are **not historically untouched**, and this report treats them as supplementary to cross-validation.

2. Objectives of the Project

The brief sets eight objectives, three of them phrased as questions (2, 3 and 5). Table 2.1 states what was done for each and where the evidence is; Sections 2.2 to 2.4 then answer the three questions directly.

2.1 The eight objectives and how each was met

#	OBJECTIVE (FROM THE BRIEF)	WHAT WAS DONE	RESULT	WHERE
1	Collect and clean real estate data	Read the CSV as text so "NA" stays visible, checked it against the brief's dictionary, split before analysis, classified every gap, ran 12 must-hold integrity rules.	All 12 must-hold rules pass; 19 predictors with NA are classified. After structural-absence rules, 302 entries remain missing or not applicable across seven columns and receive branch-specific treatment.	6.1–6.2
2	Handle ordinal and nominal columns separately, using the metadata sheet	No sheet was supplied, so one was built: all 79 predictors classified once, each assumed order tested against price.	34 numeric, 3 binary, 19 ordinal, 23 nominal; 12 orders supported, 5 not supported, 2 untestable.	2.2
3	Use scaling, PCA and fillna() for missing data and accuracy	Compared fold-fitted imputation strategies; the final pipeline uses separate imputation branches, standard scaling and clipping in its main numeric branch, and fold-fitted PCA experiments.	Scaling and the cap matter (unscaled fails); no imputer or PCA size beats the chosen design.	2.3
4	EDA to find the key variables	Univariate, outlier, bivariate and multicollinearity analysis on training rows.	28 of 33 numeric predictors associate with price; OverallQual ρ = +0.812, GrLivArea +0.730; Neighborhood ε^2 = 0.575.	6
5	The proper encoding for ordinal and nominal variables	Training-fold comparisons of encoding alternatives under documented rules, with the clipping-rule amendment disclosed.	Final candidates: declared-order ordinal codes, specified binary codes and one-hot nominal encoding. Native categorical handling was tested in a separate HistGradientBoosting experiment.	2.4
6	Develop a model that predicts price	Ten algorithms plus baselines on identical folds, then nested tuning.	Final model Lasso on log price.	8.1
7	Evaluate and compare with other algorithms	15-fold repeated CV, nested CV, paired differences, one logged held-out evaluation, bootstrap intervals.	CV R^2 0.847; held-out R^2 0.930, MAE \$12,828, RMSE \$18,512.	8.1–8.2
8	Present findings and insights clearly	This report and the Jupyter notebook.	Limitations and test-set exposure stated openly.	8.5

2.2 Objective 2: handling ordinal and nominal columns separately

Question. How can ordinal and nominal columns be handled separately, using the metadata sheet to identify which is which?

Answer. The brief refers to a metadata sheet, but none was supplied. The sheet was therefore built from the brief's descriptions, corrected where the brief's names mislead (Appendix A), and from the label sets in the training rows. Every predictor is classified exactly once, and every label in the data must sit inside its declared scale. The four groups are then routed to different encoders inside the same pipeline (Figure 3.2):

KIND	COUNT	EXAMPLES	HOW IT IS HANDLED
NUMERIC	34	GrLivArea, PropertySize, YearBuilt, OverallQual	Main numeric branch: median imputation, standard scaling and ±3 SD clipping. Frontage and garage year use separate branches.
BINARY	3	Street, CentralAir, Amenities	Coded 0 / 1 with a stated reference level
ORDINAL	19	ExterQual, KitchenQual, BsmntFinish, PropertyShape	Integer codes in the declared order, low to high; "None" (feature absent) placed below the lowest grade
NOMINAL	23	Neighborhood, PropertyClass, SaleType, Roof1Material	One-hot, one column per level; levels under 1% of rows grouped as "infrequent"

Each assumed order was checked against price: the median price must move in one direction across the levels that have at least 30 rows, and |Spearman ρ| with log price must be at least 0.10. **12 of 19 orders are supported** (for example ExterQual ρ = +0.69, KitchenQual +0.69, BsmntFinish +0.68, and PropertyShape as a falling order, ρ = −0.32). **Five are not supported** (BsmntFinRat1, BsmntFinQual1 and BoundaryFeatures zig-zag; Grade and ExterCond have |ρ| below 0.10) and **two cannot be tested** (Functional, PoolQC: fewer than two levels with 30 rows). The five unsupported orders were re-tested as one-hot columns and gave no gain (log RMSE +0.00010, Section 2.4), so no column was dropped, as the brief requires.

Two numeric-looking columns are treated as labels. PropertyClass has a numeric rank correlation with log price of ρ = −0.002 but a categorical group effect of $\varepsilon^2 = 0.361$, supporting nominal treatment rather than interpreting its codes as quantities. SaleMon is also treated as nominal. Its estimated group effect is approximately zero in this sample; this does not establish that sale-month effects are absent in other datasets.

> **So what:** The metadata step is what lets quality grades keep their order while labels such as Neighborhood do not get a false numeric meaning. Without it, PropertyClass would be mis-read as a quantity.

2.3 Objective 3: scaling, PCA and fillna() for missing data and accuracy

Question. How can scaling, PCA and fillna() be used to handle missing data and improve accuracy?

Missing data and fillna()

Missing entries require a distinction between structural absence and unknown information. Absent categorical features are represented by None where supported by the dictionary and companion columns. After these rules, 302 cells remain missing across seven columns: PropertyFrontage (217), BasementYrBlt (66, not applicable where there is no garage), ExteriorCladdingType and ExteriorCladdingArea (8 each), and BsmntVisibility, BsmntFinQual1 and Electrical (1 each). Section 3 compares imputation strategies using the

fold-fitted GapImputer. The final Section 9 pipeline implements preprocessing through separate branches: median imputation for the main numeric block and frontage, most-frequent imputation for categorical inputs, and a garage-year transformer that distinguishes no garage from an unknown year for an existing garage. Learned values are fitted within each training fold.

Scaling

Standard scaling and clipping materially improve the tested linear-model designs. In the Section 3 comparison, raw numbers worsen log RMSE by 0.0183 (95% interval +0.0104 to +0.0278). In Section 9, robust scaling increases dollar RMSE by $525 relative to standard scaling, while the unscaled variant gives RMSE $54,688. The Section 3 ridge experiment reduces RMSE from $48,635 to $31,027 with the ±3 SD guard. These are preprocessing experiments rather than the final Lasso's headline scores. In the final pipeline, standard scaling and clipping apply to the main numeric branch; PropertyFrontage and BasementYrBlt use separate branches and do not receive that branch's complete treatment.

PCA

Section 9 evaluates PCA within the main numeric branch, which excludes PropertyFrontage and BasementYrBlt and therefore contains 32 source columns. Those two variables retain their separate preprocessing branches. The 23-component PCA variant changes RMSE by −$41 (95% interval −$165 to +$83), providing no clear evidence of an improvement. Smaller variants increase RMSE: 15 components by $884, 10 by $1,130, five by $1,011 and two by $1,923. PCA is therefore documented but not adopted (Figure 6.10).

> **So what:** fillna() and scaling are part of the final pipeline; PCA is not. All three were tested the same way, on identical folds, with every fitted value learned inside the training fold.

2.4 Objective 5: the proper encoding for ordinal and nominal variables

Question. What is the proper encoding technique for ordinal and nominal variables, given the requirements of the model?

Section 3 compares preprocessing alternatives on the same 15 training folds. Its original adoption rule requires a log-RMSE reduction of at least 0.002 and a 95% difference interval below zero. The clipping choice was subsequently amended after inspecting the results, as disclosed below. Section 9 uses dollar-RMSE comparisons to check the later pipeline implementation. These are related experiments, but they should not be presented as one unchanged preprocessing design.

QUESTION	BASELINE (KEPT)	CHALLENGER AND CHANGE IN LOG RMSE	RESULT
ORDINAL COLUMNS	Codes in declared order	One-hot, order ignored −0.00072; codes plus one-hot +0.00116; unsupported five as one-hot +0.00010	Keep codes
NOMINAL COLUMNS	One-hot, every level	Levels under 10 rows pooled +0.00027; under 30 rows pooled +0.00097	Keep one-hot
MONTH SOLD	One-hot (12 columns)	Cyclic sine and cosine −0.00011	Keep one-hot
NUMBERS	Standardised	Raw +0.01831	Keep standardised
TAIL OF NUMBERS	No guard	Cap at ±5, ±4, ±3, ±2.5, ±2 SD: −0.0109, −0.0144, −0.0179, −0.0190, −0.0193	Cap at ±3 SD
NOMINAL, TREE MODELS	One-hot	Native categories −0.00271 (95% CI −0.00472 to −0.00089)	Favoured in the HistGradientBoosting probe; not used by the Section 10 candidate pipelines.
NOMINAL, SECTION 9 CHECK	Rare-grouped one-hot	Cross-fitted target encoding +$586 RMSE (CI +$380 to +$793)	Keep one-hot

The ±3 SD cap was chosen because ±2 SD scored marginally lower (0.1290 against 0.1304) but clips legitimate large houses and produced the most errors above $100,000 (44 against 34). This choice was **amended after seeing the results**, which the notebook records; it is a documented methodological amendment, not a fully pre-specified decision.

Answer. The final Section 10 candidates use the shared make_pre9 preprocessing: declared-order codes for ordinal variables, specified binary codes and one-hot encoding for nominal variables. In the recorded implementation, nominal levels occurring in fewer than 1% of training observations are grouped as infrequent. The fitted final Lasso has 200 encoded terms, of which 71 are non-zero; encoded dimensions can vary across folds and refits. Section 3 separately tests native categorical handling with HistGradientBoosting, but the Section 10 tree candidates do not use that experimental pipeline. The earlier comparison pooling levels below 10 observations is also not identical to the final 1% rule. The recorded stress tests produce finite predictions for the tested missing-value and unseen-label scenarios.

2.5 Constraints from the brief

- **No dropped columns.** All 79 predictors enter every pipeline; PropertyID is the only column kept out. Lasso sets some coefficients to zero by itself; that is model behaviour, not manual dropping.
- **R^2 target.** 0.75–0.85 expected, above 0.85 better. Reported separately for cross-validation (0.847) and the held-out rows (0.930), with the caveats in Section 1.4.
- **Metrics.** MAE and RMSE are reported in dollars alongside R^2 and log-scale RMSE.
- **Models.** The brief names linear regression, decision trees and random forests. Ten algorithms were compared, including all three.
-

3. Flow Chart of Operations

Figure 3.1 shows the analytical sequence of the twelve notebook sections. It describes the process only, not the results. The train/test split is made at the start of Section 3, before any value-based analysis; model selection uses training rows only.

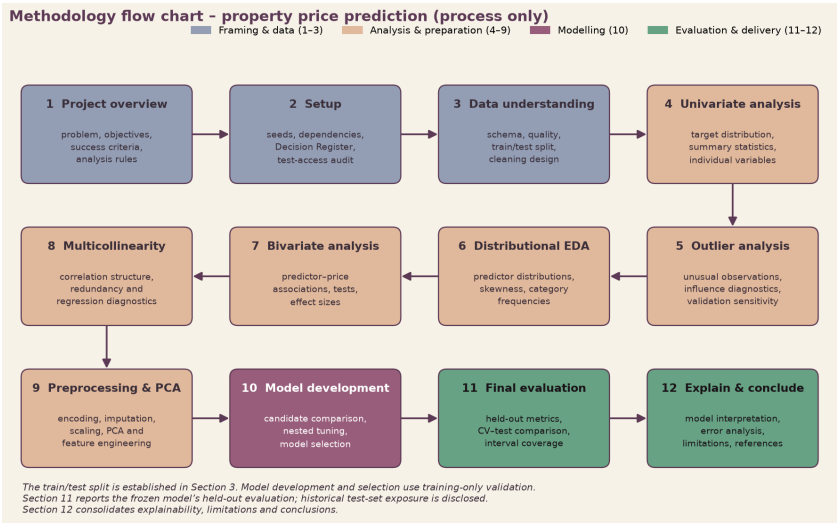


Figure 3.1. Methodology flow chart of the twelve notebook sections.

Figure 3.2 shows preprocessing within a fitted model pipeline. Learned imputation values, scaling statistics, category mappings and model coefficients are fitted on each training fold and applied to its validation fold. This controls leakage through fitted preprocessing. It does not eliminate selection optimism from repeatedly using the training dataset to choose preprocessing, shortlist models or amend analytical decisions.

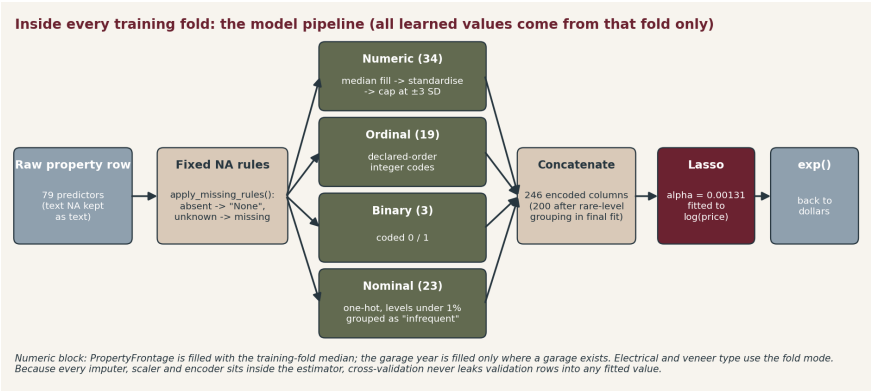


Figure 3.2. The model pipeline applied inside every training fold.

3.1 Brief methodology mapped to the work done

BRIEF STEP	WHAT WAS DONE IN THIS PROJECT	NOTEBOOK SECTION
1 DATA COLLECTION	The supplied Ames CSV was used. No scraping was done. File fingerprint (SHA-256) recorded and re-checked.	2.3, 3.1
2 DATA CLEANING	NA read as text; dictionary checked; NA classified as absent or unknown; integrity rules; fold-safe imputation.	3.1–3.7
3 EXPLORATORY DATA ANALYSIS	Univariate, outlier, distributional, bivariate and collinearity analysis.	4–8
4 FEATURE ENGINEERING	Seven domain features tested (age, years since remodel, remodel flag, total area, outdoor area, total bathrooms, quality × area). None met the adoption rule.	9.7b
5 MODEL SELECTION	Ten algorithms plus baselines compared on identical folds; nested tuning of the best three.	10
6 TRAINING AND TESTING	Random 80/20 split; repeated 5 × 3 cross-validation on training rows; one logged held-out evaluation.	3.4, 10, 11
7 EVALUATION	R^2, MAE, RMSE (dollars) and log RMSE; bootstrap intervals; subgroup errors; 80% conformal intervals.	11
8 DEPLOYMENT	The fitted Lasso pipeline was saved with joblib (32.7 KB) and reloaded in an independent Python process with maximum prediction difference $0.00. Production monitoring and external validation were not done.	11.1

4. Python Codes

This section reproduces the key code of the project, copied from the Jupyter notebook and abridged only by removing table-display, plotting and logging lines. The complete, runnable code is in the notebook, which is submitted with this report. The full environment (Python 3.14.3 and the library versions) is recorded by the notebook in requirements.txt and RUN_INSTRUCTIONS.md.

> **Reading guide:** Names ending in a section number (for example X9, CANDIDATES10, FINAL_ESTIMATOR11) are notebook objects created in that section. X9 and y9 are the 1,168 cleaned training rows and their prices; the held-out rows stay inside a logged TestVault until Section 11.

4.1 Reading the data and splitting it

Code 4.1 Read every cell as text so that “NA” stays visible (notebook Section 3.1)



```
# — Read the file as text, after checking it is the file fingerprinted in 2.3 —
ID_COL, TARGET = 'PropertyID', 'PropPrice'

assert sha256_of(DATA_PATH) == DATA_SHA256, 'The data file has changed since Section 2.3.'

# dtype=str + keep_default_na=False: nothing is converted yet, so the literal text "NA" stays visible
RAW = pd.read_csv(DATA_PATH, dtype=str, keep_default_na=False, na_values=[])

def parses_as_number(s):
    try:
        return bool(np.isfinite(float(s)))
    except ValueError:
        return False

def token_audit(raw):
    """Per column: does it contain the text NA, is the rest numeric, any blank cells, any padded cells?
    Presence only. No counts, proportions or other value statistics are computed before the split."""
    rows = []
    for col in raw.columns:
        v = raw[col]
        rest = v[v != 'NA']
        rows.append({'Column': col,
                     'Has NA text': bool((v == 'NA').any()),
                     'Rest is numeric': bool(rest.map(parses_as_number).all()),
                     'Has blank cells': bool((v.str.strip() == '').any()),
                     'Has padded cells': bool((v != v.str.strip()).any())})
    return pd.DataFrame(rows)

AUDIT = token_audit(RAW)

na_numeric = AUDIT.loc[AUDIT['Has NA text'] & AUDIT['Rest is numeric'], 'Column'].tolist()
na_text    = AUDIT.loc[AUDIT['Has NA text'] & ~AUDIT['Rest is numeric'], 'Column'].tolist()

checks = pd.DataFrame([
    ('File is the one fingerprinted in 2.3 (SHA-256 equal)',          '✅'),
    ('First column is the ID and the last is the target',             '✅' if (RAW.columns[0], RAW.columns[-1]) == (ID_COL, TARGET) else '❌'),
    ('Rows × columns',                                                f'{RAW.shape[0]:,} × {RAW.shape[1]}'),
    ('Predictors (all columns except ID and target)',                 f'{RAW.shape[1] - 2}'),
    ('Columns whose other values are all numbers',                    f'{int(AUDIT["Rest is numeric"].sum())} (including ID and target)'),
    ('Columns that contain the text NA',                              f'{len(na_numeric) + len(na_text)}  ({len(na_numeric)} numeric-looking, {len(na_text)} text)'),
    ('Columns with blank cells',                                      f'{int(AUDIT["Has blank cells"].sum())}'),
    ('Columns with leading / trailing spaces',                        f'{int(AUDIT["Has padded cells"].sum())}'),
], columns=['Check', 'Result'])
show_table(checks, caption='Reading the file')

print('\nNumeric-looking columns that carry the text NA :', ', '.join(na_numeric))
print('Text columns that carry the text NA            :', ', '.join(na_text))

DECISIONS.add('3.1', 'How is the CSV read so that the text "NA" is not silently turned into a missing value?',
    'Read every cell as text (dtype=str, keep_default_na=False). Audit each column for the text NA, blanks and padding. Presence only: no counts or value statistics before the split.',
    f'Fingerprint unchanged. {RAW.shape[0]:,} × {RAW.shape[1]}. The text NA appears in {len(na_numeric) + len(na_text)} columns ({len(na_numeric)} numeric-looking: {", ".join(na_numeric)}). '
    f'Blank cells in {int(AUDIT["Has blank cells"].sum())} columns; padded cells in {int(AUDIT["Has padded cells"].sum())}.',
    'Numeric columns are converted explicitly (NA becomes a missing number). In text columns NA stays a visible label until 3.5 decides, column by column, whether it means absent or unknown.',
    'Changes the workflow')
print('\n✅ Raw file read as text; NA kept visible. Decision registered.')
```

Code 4.2 Reproducible 80/20 split with a fingerprint of the test IDs (notebook Section 3.4)



```
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.dummy import DummyClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer

if 'SP_ALL' not in globals():
    raise RuntimeError('SP_ALL no longer exists: the split was already made in this session. Restart the kernel and run 3.1 to 3.4 in order.')

# — 1. The split: a plain random 80 / 20 split of the rows —
# The target is NOT used to build the split, so test labels stay unseen until the final evaluation (Section 11).
TEST_FRACTION = 0.20
all_ids = SP_ALL[ID_COL].to_numpy()
train_ids, test_ids = train_test_split(all_ids, test_size=TEST_FRACTION, random_state=DEEDS['split'], shuffle=True)
train_ids, test_ids = np.sort(train_ids), np.sort(test_ids)

FULL    = SP_ALL.set_index(ID_COL)
X_train = FULL.loc[train_ids].drop(columns=TARGET)
y_train = FULL.loc[train_ids, TARGET]
VAULT   = TestVault(FULL.loc[test_ids].drop(columns=TARGET), FULL.loc[test_ids, TARGET], TEST_LOG)

SPLIT_FINGERPRINT = hash(is_shuffled=', test_ids=).encode()).hexdigest()
print(f'Split: {len(train_ids):,} train / {len(test_ids):,} test  (test fraction {len(test_ids) / (len(train_ids) + len(test_ids)):.2f})')
print(f'Test-ID fingerprint (SHA-256, first 16): {SPLIT_FINGERPRINT[:16]}…')
assert_valid_ids(ID_COL, np.concatenate([train_ids, test_ids]), ALL_IDS, 'train + test')

# From here on nothing outside the vault may hold test rows: the full frames are deleted.
n_total = len(SP_ALL)
del SP_ALL, FULL, RAW

split_checks = pd.DataFrame([
    ('Rows: train + test = all rows',          f'{len(train_ids):,} + {len(test_ids):,} = {len(train_ids) + len(test_ids):,}'),
    ('Test share',                             f'{len(test_ids) / n_total:.3f}'),
    ('No PropertyID in both parts',            '✅' if not set(train_ids) & set(test_ids) else '❌'),
    ('Every PropertyID is in one part',        '✅' if len(set(train_ids) | set(test_ids)) == n_total else '❌'),
    ('Seed used',                              f"{DEEDS['split']} = {DEEDS['split']}"),
    ('Test-ID fingerprint (SHA-256, first 16)', SPLIT_FINGERPRINT[:16]),
    ('Train rows × columns',                   f'{X_train.shape[0]:,} × {X_train.shape[1]}, y_train ({len(y_train):,},)'),
    ('Test rows',                              f'locked in VAULT only ({len(VAULT):,} rows)'),
], columns=['Item', 'Result'])
show_table(split_checks, caption='The train / test split')
assert not set(train_ids) & set(test_ids) and len(set(train_ids) | set(test_ids)) == n_total

# — 2. Adversarial validation: can a model tell train rows from test rows, using the predictors only? —
# If it can, the split is not random with respect to the predictors. Section 3.5 investigates why.
X_both = pd.concat([X_train, VAULT.X_test])
X_both['split_origin_feature'] = (['train'] * len(X_train) + ['test'] * len(VAULT.X_test))
X_both = pd.concat([X_train, X_test_diag])
cv_test = cv_3by_survive(X_train, shuffle=True, random_state=DEEDS['cv'])
num_cols = [c for c in X_both.columns if c in NUMERIC_PROS]
cat_cols = [c for c in X_both.columns if c in TEXT_PROS]

adv_model = Pipeline([
    ('prep', ColumnTransformer([('num', SimpleImputer(strategy='median'), num_cols),
                                ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)],
                               remainder='drop', sparse_threshold=0)),
    ('clf',  RandomForestClassifier(n_estimators=200, min_samples_leaf=5, class_weight='balanced',
                                    random_state=DEEDS['rf'], n_jobs=-1))])
adv_cv  = StratifiedKFold(n_splits=5, shuffle=True, random_state=DEEDS['cv'])
adv_auc = cross_val_score(adv_model, X_both, is_test, cv=adv_cv, scoring='roc_auc')

# Null: the same procedure on shuffled train / test labels, so a chance-level AUC has a reference range
N_PERM = 40
rng = np.random.default_rng(DEEDS['permutation'])
null_auc = np.array([cross_val_score(adv_model, X_both, rng.permutation(is_test), cv=adv_cv, scoring='roc_auc').mean() for _ in range(N_PERM)])
null_lo, null_hi = np.percentile(null_auc, [2.5, 97.5])
p_perm = (1 + int((null_auc >= adv_auc.mean()).sum())) / (N_PERM + 1)
split_accepted = p_perm >= 0.05                  # the rule was fixed before the run (see the Decision above)

adv_table = pd.DataFrame([
    ('Observed cross-validated AUC (mean ± SD over 5 folds)', f'{adv_auc.mean():.3f} ± {adv_auc.std(ddof=1):.3f}'),
    ('Chance-level AUC from shuffled labels (mean)',          f'{null_auc.mean():.3f}'),
    ('Range of shuffled-label AUC (2.5 % to 97.5 %)',         f'{null_lo:.3f} to {null_hi:.3f}'),
    ('Permutation p-value (N_PERM shuffles)',                 f'{p_perm:.3f}'),
    ('Below the 5 % level?  (p < 0.05)',                      'accepted' if split_accepted else 'NOT accepted: investigate'),
], columns=['Item', 'Result'])
show_table(adv_table, caption='Adversarial validation: train vs test, predictors only')

fig, ax = plt.subplots(figsize=(8.5, 3.8))
ax.hist(null_auc, bins=15, color='#cccccc', edgecolor='#888888', label='Shuffled labels (N_PERM runs)')
ax.axvline(adv_auc.mean(), color='#B22222', lw=2, label=f'Observed AUC = {adv_auc.mean():.3f}')
ax.set_title('Adversarial validation: can a model tell train rows from test rows?', loc='left')
ax.set_xlabel('Cross-validated AUC  (0.5 = cannot tell them apart)'); ax.set_ylabel('Number of runs')
ax.legend(frameon=False, loc='upper right')
save_fig(fig, '03_adversarial_validation', section='3')

# — 3. Labels that exist in the test rows but never appear in training —
rows = []
for c in cat_cols:
    unseen = sorted(set(X_test_diag[c]) - set(X_train[c]))
    if unseen:
        rows.append({'Column': c, 'Labels only in test': ', '.join(unseen), 'Test rows affected': int(X_test_diag[c].isin(unseen).sum())})
UNSEEN = pd.DataFrame(rows, columns=['Column', 'Labels only in test', 'Test rows affected'])
if UNSEEN.empty:
    print('Labels only in the test rows: none.')
else:
    show_table(UNSEEN, caption='Labels that never appear in the training rows')
n_unseen_rows = int(X_test_diag[UNSEEN['Column'].tolist()].apply(lambda col: col.isin(set(col) - set(X_train[col.name]))).any(axis=1).sum()) if len(UNSEEN) else 0
del X_test_diag, X_both                          # the test predictors do not stay in memory outside the vault

# — Decisions —
DECISIONS.add('3.4', 'How is the data split, and does the split hide anything?',
    f'A plain random {1 - TEST_FRACTION:.0%} / {TEST_FRACTION:.0%} split of the rows with SEEDS["split"]. The target is not used to build it. The test rows go into TestVault and the full frames are deleted.',
    f'{len(train_ids):,} training and {len(test_ids):,} test rows, no overlap. Test-ID fingerprint ({SPLIT_FINGERPRINT[:16]}…). split_ids.csv is saved to artifacts/.',
    'All value-based work from 3.5 onward uses X_train and y_train. The test rows are opened once more, for the final evaluation (Section 11). Because the target did not shape the split, test and training prices are not compared until then, and that is a stated limit, not an oversight.',
    'Changes the workflow')
DECISIONS.add('3.4', 'Do the training and test rows look alike on the predictors?',
    'Adversarial validation: a model tries to tell train rows from test rows using the predictors only. The 5-fold AUC is compared with an AUC from 40 shuffled-label runs. Rule fixed in advance: accept the split if the permutation p ≥ 0.05; if not, find which predictor differs instead of re-drawing the seed.',
    f'Observed AUC {adv_auc.mean():.3f} ± {adv_auc.std(ddof=1):.3f}; shuffled-label AUC {null_auc.mean():.3f} (2.5 % to 97.5 %: {null_lo:.3f} to {null_hi:.3f}); permutation p = {p_perm:.3f}.',
    'Reading the test predictors was logged as a split-diagnostics access (target not used).',
    f'Split accepted: {split_accepted}' if split_accepted else 'Split NOT accepted: investigate before 3.5.',
    'Deliberately unchanged',
DECISIONS.add('3.4', 'Do the test rows contain labels the training rows never show?',
    'For each text predictor, list labels present in the test predictors but absent from the training rows, and the number of test rows affected.',
    'Labels only in the test rows: none.' if UNSEEN.empty else f'{len(UNSEEN)} column(s) carry labels seen only in the test rows, affecting {n_unseen_rows} test rows: ' + '; '.join(f'{r.Column} ({r._2}: {r._3})' for r in UNSEEN.itertuples()) + '.',
    'The action is needed now.' if UNSEEN.empty else 'Every encoder in 3.10 and in the pipeline must ignore unseen labels instead of failing (handle_unknown="ignore"), and these rows are watched in the final evaluation.',
    'Documentation only' if UNSEEN.empty else 'Changes the workflow')

print('\nTest-access log:')
TEST_LOG.show()
print(f'\n✅ Split made, checked and logged. Section 3 decisions so far: {len(DECISIONS.to_frame("D"))}')
```

4.2 Missing values: absent versus unknown

Code 4.3 Companion-column rules and `apply_missing_rules()` (notebook Section 3.5)



Code 4.4 `GapImputer`: fold-safe fillna() (notebook Section 3.7)





4.3 Metadata sheet and preprocessing pipeline

Code 4.5 Ordinal scales and binary coding: the metadata sheet in code (notebook Section 3.9)



Code 4.6 Importable module with the tail guard and the garage-year imputer (notebook Section 9.0)

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt, seaborn as sns
from sklearn.base import BaseEstimator, TransformerMixin, clone
from sklearn.compose import ColumnTransformer, TransformedTargetRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler, RobustScaler, FunctionTransformer
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.linear_model import Ridge
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error
from sklearn.decomposition import PCA
from scipy import stats

X9 = X_train_clean.copy(); y9 = y_train.astype(float).copy()
assert X9.index.equals(y9.index) and (y9 > 0).all()
GROUPS9 = {'numeric':list(NUM_COLS), 'ordinal':list(ORD_COLS), 'binary':list(BIN_COLS), 'nominal':list(NOM_COLS)}
assert len(sum(GROUPS9.values(),[])) == len(set(sum(GROUPS9.values(),[]))), 'Metadata groups overlap'
assert set(sum(GROUPS9.values(),[])) == set(X9.columns), 'Metadata does not cover source predictors'
assert 'PropertyID' not in X9 and 'PropPrice' not in X9
CV9 = list(RepeatedKFold(n_splits=5,n_repeats=3,random_state=SEEDS['repeated_cv']).split(X9))
print('Training:',X9.shape,'groups:',{k:len(v) for k,v in GROUPS9.items()})
print('CV:',[(len(tr),len(va)) for tr,va in CV9]); print('Test vault untouched')
```

Code 4.7 Preprocessing: scaling, tail guard, ordinal, binary and one-hot blocks (notebook Section 9)

9.9 Decision Register and Section 9 Summary

Research question. Which preprocessing and dimensionality-reduction choices are supported by the training-only validation experiments?

Method. Consolidate the paired cross-validation comparisons covering imputation, missingness indicators, ordinal encoding, scaling, skewness treatment, PCA and engineered features. Record the selected configurations and supporting evidence in the Decision Register.

Evidence. The comparisons use repeated cross-validation within the training partition. Their results provide evidence about relative preprocessing performance under the evaluated folds, but repeated experimentation on the same validation structure can introduce model-selection optimism.

Decision. Treat the Section 9 choices as **validation-supported modelling decisions**, not as independently confirmed improvements. The final candidate pipelines are evaluated through the model-development and tuning procedures in Section 10. The held-out test results in Section 11 are reported separately and are not used to retrospectively choose preprocessing configurations.

```
for sec,q,tab,decision in [
    ('9.1','Which frontage imputation strategy?',imputation9,'Fold-fitted median; no fill beats it beyond noise.'),
    ('9.2','Does missing frontage carry signal?',indicator9,'No missing indicator; it is slightly worse.'),
    ('9.3','Ordinal grades: integer or one-hot?',ordinal9,'Metadata ordinal codes; one-hot grades give no gain.'),
    ('9.5','Which scaling for Ridge?',scaling9,'Standard scaling for the linear family; robust is worse and unscaled fails without the tail guard.'),
    ('9.6','Does area log1p help?',skew9,'No log1p; no gain over the original columns.'),
    ('9.7','Does numeric PCA help?',pca9,'PCA not adopted; fewer than 23 components cost accuracy.')]:
    best=tab.iloc[0]
    DECISIONS.add(sec,q,'Paired repeated 5-fold CV (15 folds), dollar-scale RMSE',
                  f"Best: {best['Candidate']} RMSE ${best['Mean RMSE ($)']:,.0f}; paired interval in subsection table.",
                  decision,'Deliberately unchanged')
DECISIONS.add('9.4','How to encode nominal columns?', 'Fold-fitted one-hot versus internally cross-fitted target encoding',
              'Comparison executed' if nominal9 is not None else 'TargetEncoder unavailable; one-hot baseline retained',
              'Rare-grouped one-hot; target encoding is worse.','Deliberately unchanged')
DECISIONS.add('9.8','Are learned preprocessing steps fold-confined?', 'Pipeline contract and leakage assertions',
              f"{audit9_df['Pass'].sum()}/{len(audit9_df)} checks passed, including a behavioural test of the fitted scaler",
              'No test access; fit each transformer inside CV.','Documentation only')
DECISIONS.show(section='9')
```

Python

ID	Section	Question	Method	Evidence	Decision	Effect
D61	9.7b	Do domain-derived features improve prediction beyond the original columns?	Seven row-wise features (age, years since remodel, remodel flag, total area, outdoor area, total bathrooms, quality × area) built inside the pipeline; each group alone and all together, same 15 folds, ridge probe and tail guard, same rule and threshold as 3.10.	+ Age features: +0.00003; + Total living area: -0.00077; + Outdoor area: +0.00010; + Total bathrooms: +0.00010; + Quality × area: -0.00042; + All engineered: -0.00083.	None adopted; the original columns are retained unchanged.	Deliberately unchanged
D62	9.1	Which frontage imputation strategy?	Paired repeated 5-fold CV (15 folds), dollar-scale RMSE	Best: KNN (5) RMSE $30,842; paired interval in subsection table.	Fold-fitted median; no fill beats it beyond noise.	Deliberately unchanged
D63	9.2	Does missing frontage carry signal?	Paired repeated 5-fold CV (15 folds), dollar-scale RMSE	Best: No indicator RMSE $30,992; paired interval in subsection table.	No missing indicator; it is slightly worse.	Deliberately unchanged
D64	9.3	Ordinal grades: integer or one-hot?	Paired repeated 5-fold CV (15 folds), dollar-scale RMSE	Best: Metadata ordinal RMSE $30,992; paired interval in subsection table.	Metadata ordinal codes; one-hot grades give no gain.	Deliberately unchanged
D65	9.5	Which scaling for Ridge?	Paired repeated 5-fold CV (15 folds), dollar-scale RMSE	Best: Standard RMSE $30,992; paired interval in subsection table.	Standard scaling for the linear family; robust is worse and unscaled fails without the tail guard.	Deliberately unchanged
D66	9.6	Does area log1p help?	Paired repeated 5-fold CV (15 folds), dollar-scale RMSE	Best: No log1p RMSE $30,992; paired interval in subsection table.	No log1p; no gain over the original columns.	Deliberately unchanged
D67	9.7	Does numeric PCA help?	Paired repeated 5-fold CV (15 folds), dollar-scale RMSE	Best: PCA 23 RMSE $30,951; paired interval in subsection table.	PCA not adopted; fewer than 23 components cost accuracy.	Deliberately unchanged
D68	9.4	How to encode nominal columns?	Fold-fitted one-hot versus internally cross-fitted target encoding	Comparison executed	Rare-grouped one-hot; target encoding is worse.	Deliberately unchanged
D69	9.8	Are learned preprocessing steps fold-confined?	Pipeline contract and leakage assertions	7/7 checks passed, including a behavioural test of the fitted scaler	No test access; fit each transformer inside CV.	Documentation only

Code 4.8 PCA experiment on the numeric block (notebook Section 9.7)

```
numeric9=[c for c in NUM_COLS if c not in ('PropertyFrontage','BasementYrBlt')]
num9=SimpleImputer(strategy='median').fit_transform(X9[numeric9])
num9=StandardScaler().fit_transform(num9)
pca_descriptive9=PCA().fit(num9)
cum9=np.cumsum(pca_descriptive9.explained_variance_ratio_)
fig,ax=plt.subplots(figsize=(10,5));ax.plot(np.arange(1,len(cum9)+1),cum9,marker='o',markersize=3)
ax.axhline(.90,linestyle='--',color='#985D7B',label='90% variance');ax.axhline(.95,linestyle=':',color='#67A284',label='95% variance')
ax.set(xlabel='Numeric PCA components',ylabel='Cumulative variance explained',title='Figure 9.1 — Training-only numeric PCA scree');ax.legend();plt.tight_layout()
save_fig(fig,'fig9_1_numeric_pca_scree',section='9');plt.show()
print('k at 90%:',np.searchsorted(cum9,.90)+1,'k at 95%:',np.searchsorted(cum9,.95)+1)
ks9=sorted(set([2,5,10,min(15,len(numeric9)),int(np.searchsorted(cum9,.95)+1)]))
ks9=[k for k in ks9 if 1<=k<=len(numeric9)]
pca_models9={'No PCA':model9()}
pca_models9.update({f'PCA {k}':model9(pca=k) for k in ks9})
pca9,pca_folds9=compare9(pca_models9,'No PCA')
fig,ax=plt.subplots(figsize=(9,4.5));ax.bar(pca9['Candidate'],pca9['Mean RMSE ($)'],color='#985D7B');ax.tick_params(axis='x',rotation=35)
ax.set(ylabel='Five-fold CV RMSE ($)',title='Figure 9.2 — PCA ablation (training CV)');plt.tight_layout()
save_fig(fig,'fig9_2_pca_cv',section='9');plt.show()
```

4.4 Model comparison, tuning and final evaluation

Code 4.9 Metrics, cross-validated evaluation and the log-target wrapper (notebook Section 10.1)

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from scipy import stats
from sklearn.base import clone
from sklearn.model_selection import RepeatedKFold, KFold, RandomizedSearchCV
from sklearn.pipeline import Pipeline
from sklearn.compose import TransformedTargetRegressor
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR
from sklearn.ensemble import RandomForestRegressor, ExtraTreesRegressor, GradientBoostingRegressor, VotingRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

assert 'X9' in globals() and 'y9' in globals() and callable(make_pre9)
assert X9.index.equals(y9.index) and (y9 > 0).all()
assert 'PropertyID' not in X9.columns and 'PropPrice' not in X9.columns
SEED10=int(SEEDS.get('repeated_cv',2026)); INNER_SEED10=int(SEEDS.get('tuning',7))
CV10=list(RepeatedKFold(n_splits=5,n_repeats=3,random_state=SEED10).split(X9,y9))
FOLD_AUDIT10=pd.DataFrame([{'Fold':i+1,'Train n':len(tr),'Validation n':len(va),
    'Validation price min ($)':float(y9.iloc[va].min()),'Validation price max ($)':float(y9.iloc[va].max()),
    'Overlap':len(set(tr)&set(va))} for i,(tr,va) in enumerate(CV10)])
assert FOLD_AUDIT10['Overlap'].eq(0).all()
print('Repeated CV: 5 folds × 3 repeats; seed:',SEED10,'| inner seed:',INNER_SEED10)
display(FOLD_AUDIT10)

def metrics10(y,p):
    p=np.asarray(p,dtype=float); y=np.asarray(y,dtype=float)
    assert np.isfinite(p).all()
    return {'RMSE ($)':float(np.sqrt(mean_squared_error(y,p))),
            'MAE ($)':float(mean_absolute_error(y,p)),
            'R²':float(r2_score(y,p)),
            'RMSE (log)':float(np.sqrt(mean_squared_error(np.log(y),np.log(np.maximum(p,1e-9))))) if (p>0).all() else np.nan}

def evaluate10(models,folds=CV10,X=X9,y=y9):
    rows=[]
    for name,est in models.items():
        for k,(tr,va) in enumerate(folds):
            fit=clone(est).fit(X.iloc[tr],y.iloc[tr]); pred=fit.predict(X.iloc[va])
            rows.append({'Model':name,'Fold':k+1,**metrics10(y.iloc[va],pred)})
    return pd.DataFrame(rows)

def summary10(scores):
    return scores.groupby('Model',as_index=False).agg(
        RMSE_mean=('RMSE ($)','mean'),RMSE_sd=('RMSE ($)','std'),
        MAE_mean=('MAE ($)','mean'),R2_mean=('R²','mean'),
        log_RMSE_mean=('RMSE (log)','mean')).sort_values('RMSE_mean').reset_index(drop=True)

def log_model10(est,**prekw):
    return TransformedTargetRegressor(regressor=Pipeline([
        ('pre',make_pre9(**prekw)),('model',est)]),func=np.log,inverse_func=np.exp)
```

Code 4.10 The ten candidate algorithms (notebook Section 10.3)

```
from sklearn.tree import DecisionTreeRegressor
CANDIDATES10={
 'Ridge':log_model10(Ridge(alpha=100.)),
 'Linear Regression':log_model10(LinearRegression()),
 'Decision Tree':log_model10(DecisionTreeRegressor(max_depth=6,min_samples_leaf=10,random_state=SEED10)),
 'Lasso':log_model10(Lasso(alpha=.001,max_iter=200000,selection='random',random_state=SEED10)),
 'ElasticNet':log_model10(ElasticNet(alpha=.001,l1_ratio=.5,max_iter=200000,selection='random',random_state=SEED10)),
 'KNN':log_model10(KNeighborsRegressor(n_neighbors=12,weights='distance')),
 'SVR':log_model10(SVR(C=10.,epsilon=.05)),
 'Random Forest':log_model10(RandomForestRegressor(n_estimators=150,min_samples_leaf=2,n_jobs=-1,random_state=SEED10)),
 'ExtraTrees':log_model10(ExtraTreesRegressor(n_estimators=150,min_samples_leaf=2,n_jobs=-1,random_state=SEED10)),
 'Gradient Boosting':log_model10(GradientBoostingRegressor(n_estimators=200,learning_rate=.05,max_depth=3,random_state=SEED10))
}
# This cell can take several minutes, especially for the 15 × 8 tree-model fits.
CANDIDATE_SCORES10=evaluate10(CANDIDATES10)
CANDIDATE_SUMMARY10=summary10(CANDIDATE_SCORES10)
display(CANDIDATE_SUMMARY10.round(3))
fig,ax=plt.subplots(figsize=(11,5))
order=CANDIDATE_SUMMARY10['Model'].tolist()
ax.boxplot([CANDIDATE_SCORES10.loc[CANDIDATE_SCORES10.Model.eq(m),'RMSE ($)'] for m in order],
           tick_labels=order,showfliers=True)
ax.tick_params(axis='x',rotation=35);ax.set_ylabel('Validation RMSE ($)')
ax.set_title('Figure 10.1 — Repeated-CV dollar RMSE by algorithm')
plt.tight_layout()
try:save_fig(fig,'fig10_1_candidate_cv_rmse',section='10')
except NameError:pass
plt.show()
SHORTLIST10=CANDIDATE_SUMMARY10['Model'].head(3).tolist()
print('Exploratory shortlist (to be confirmed in nested CV):',SHORTLIST10)
```

Code 4.11 Nested hyperparameter tuning (notebook Section 10.4)

```
from scipy.stats import loguniform, randint, uniform
PARAMS10={
 'Ridge':{'regressor__model__alpha':loguniform(.01,1000)},
 'Lasso':{'regressor__model__alpha':loguniform(1e-5,.05)},
 'ElasticNet':{'regressor__model__alpha':loguniform(1e-5,.05),
               'regressor__model__l1_ratio':uniform(.1,.85)},
 'KNN':{'regressor__model__n_neighbors':randint(3,30),
        'regressor__model__weights':['uniform','distance']},
 'SVR':{'regressor__model__C':loguniform(.1,100),
        'regressor__model__epsilon':loguniform(.005,.2)},
 'Random Forest':{'regressor__model__max_features':[.5,.75,1.],
       'regressor__model__min_samples_leaf':randint(1,6),
       'regressor__model__max_depth':[None,10,20]},
 'ExtraTrees':{'regressor__model__max_features':[.5,.75,1.],
       'regressor__model__min_samples_leaf':randint(1,6),
       'regressor__model__max_depth':[None,10,20]},
 'Gradient Boosting':{'regressor__model__learning_rate':loguniform(.02,.15),
       'regressor__model__n_estimators':randint(100,400),
       'regressor__model__max_depth':randint(2,5)}
}
# Nested outer CV is distinct from the repeated exploratory comparison above.
OUTER10=list(KFold(n_splits=5,shuffle=True,random_state=SEED10).split(X9,y9))
INNER10=KFold(n_splits=3,shuffle=True,random_state=INNER_SEED10)
N_ITER10 = 10  # Fixed, documented computational search budget
nested_rows10=[]; nested_settings10=[]
for name in SHORTLIST10:
    for fold,(tr,va) in enumerate(OUTER10,1):
        search=RandomizedSearchCV(clone(CANDIDATES10[name]),PARAMS10[name],n_iter=N_ITER10,
            scoring='neg_root_mean_squared_error',cv=INNER10,random_state=INNER_SEED10+fold,
            n_jobs=1,refit=True,error_score='raise')
        search.fit(X9.iloc[tr],y9.iloc[tr])
        p=search.predict(X9.iloc[va])
        nested_rows10.append({'Model':name,'Fold':fold,**metrics10(y9.iloc[va],p)})
        nested_settings10.append({'Model':name,'Fold':fold,'Best parameters':search.best_params_,
                                  'Inner best RMSE ($)':-search.best_score_})
NESTED_SCORES10=pd.DataFrame(nested_rows10)
NESTED_SETTINGS10=pd.DataFrame(nested_settings10)
NESTED_SUMMARY10=summary10(NESTED_SCORES10)
display(NESTED_SUMMARY10.round(3));display(NESTED_SETTINGS10)
```

Code 4.12 Freeze the final Lasso using training rows only (notebook Section 11.1)

```
# ── 11.1 Final Model Freeze and Held-Out Test Access ──────────────

# ── Imports ──────────────────────────────────────────────────────
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import hashlib
import json
import warnings
from scipy import stats
from sklearn.base import clone
from sklearn.model_selection import KFold, RandomizedSearchCV, train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.inspection import permutation_importance
from statsmodels.stats.diagnostic import het_breuschpagan
import statsmodels.api as sm

# ── Validate required objects from earlier sections ──────────────
required_objects11 = ('X9', 'y9', 'VAULT', 'TEST_LOG', 'CANDIDATES10', 'PARAMS10', 'INNER_SEED10', 'SEEDS')
missing_objects11 = [name for name in required_objects11 if name not in globals()]
assert not missing_objects11, (f'Missing required objects: {missing_objects11}')
assert X9.index.equals(y9.index), ('Training feature and target indices do not match.')
assert (y9 > 0).all(), ('Target must contain strictly positive prices.')
assert TEST_LOG.count('final evaluation') == 0, ('Final test already accessed. Do not rerun Section 11.')

# ── Freeze the approved Section 10 winner ────────────────────────
# Chosen from training-only nested CV (10.4) under the rule written in 10.5.
# No test data are used in this step.
FINAL_MODEL_NAME11 = 'Lasso'

# Tune on ALL training rows (inner CV only), then freeze the best parameters.
FINAL_SEARCH11 = RandomizedSearchCV(
    clone(CANDIDATES10[FINAL_MODEL_NAME11]), PARAMS10[FINAL_MODEL_NAME11], n_iter=40,
    scoring='neg_root_mean_squared_error', cv=KFold(5, shuffle=True, random_state=INNER_SEED10),
    random_state=INNER_SEED10, n_jobs=1, refit=False, error_score='raise')
FINAL_SEARCH11.fit(X9, y9)
FINAL_MODEL_PARAMS11 = dict(FINAL_SEARCH11.best_params_)
print('Frozen parameters (training-only search):', FINAL_MODEL_PARAMS11,
      f'| inner-CV RMSE ${-FINAL_SEARCH11.best_score_:,.0f}')

# ── Validate model selection ─────────────────────────────────────
assert FINAL_MODEL_NAME11 in CANDIDATES10, (f'{FINAL_MODEL_NAME11!r} is not available in CANDIDATES10.')
assert isinstance(FINAL_MODEL_PARAMS11, dict)

# ── Construct the frozen estimator ───────────────────────────────
FINAL_ESTIMATOR11 = clone(CANDIDATES10[FINAL_MODEL_NAME11])
if FINAL_MODEL_PARAMS11:
    FINAL_ESTIMATOR11.set_params(**FINAL_MODEL_PARAMS11)

EXPECTED_CLASS11 = {'Lasso': 'Lasso', 'ElasticNet': 'ElasticNet', 'Gradient Boosting': 'GradientBoostingRegressor'}
assert (type(FINAL_ESTIMATOR11.regressor.named_steps['model']).__name__
        == EXPECTED_CLASS11[FINAL_MODEL_NAME11]), 'The selected estimator does not match FINAL_MODEL_NAME11.'

# ── Record frozen model configuration ────────────────────────────
FREEZE_SPEC11 = {
    'model': FINAL_MODEL_NAME11, 'params': FINAL_ESTIMATOR11.get_params(deep=True),
    'tuning_inner_cv_rmse': float(-FINAL_SEARCH11.best_score_),
    'split_seed': SEEDS.get('split'), 'cv_seed': SEEDS.get('repeated_cv'),
    'n_training': len(X9), 'training_columns': list(X9.columns)
}

# ── Convert configuration into serialisable values ───────────────
def _safe11(obj):
    if isinstance(obj, (str, int, float, bool, type(None))): return obj
    if isinstance(obj, (list, tuple)): return [_safe11(x) for x in obj]
    if isinstance(obj, dict): return {str(k): _safe11(v) for k, v in obj.items()}
    return repr(obj)
```

```
# ── Generate configuration fingerprint ───────────────────────────
FREEZE_SHA11 = hashlib.sha256(json.dumps(_safe11(FREEZE_SPEC11), sort_keys=True).encode()).hexdigest()

# ── Report frozen model configuration ────────────────────────────
print('FROZEN MODEL CONFIGURATION')
print(f'Model: {FINAL_MODEL_NAME11}')
print(f'Configuration SHA256: {FREEZE_SHA11}')
print(f'Explicit parameter overrides: {FINAL_MODEL_PARAMS11}')
print(f'\n{FINAL_MODEL_NAME11} hyperparameters:')
model_params11 = FINAL_ESTIMATOR11.regressor.named_steps['model'].get_params()
for parameter, value in sorted(model_params11.items()):
    print(f'  {parameter}: {value}')

# ── Fit final estimator on complete training data ────────────────
FINAL_FITTED11 = clone(FINAL_ESTIMATOR11).fit(X9, y9)

# ── Final pre-access safeguards ──────────────────────────────────
assert TEST_LOG.count('final evaluation') == 0, ('Held-out test was accessed during model preparation.')

# ── Sole primary access to held-out test data ────────────────────
X_TEST11, y_TEST11 = VAULT.open(
    '11.1', 'final evaluation',
    (f'Locked final model {FINAL_MODEL_NAME11}; ' f'config SHA256 {FREEZE_SHA11}'))

# ── Validate train/test feature schema ───────────────────────────
assert X_TEST11.columns.tolist() == X9.columns.tolist(), ('Train/test feature schema mismatch.')
assert X_TEST11.index.is_unique, ('Held-out test indices must be unique.')
assert not X9.index.intersection(X_TEST11.index).size, ('Training and held-out test indices overlap.')
assert len(X_TEST11) == len(y_TEST11), ('Held-out features and targets have different row counts.')

# ── Verify final evaluation access count ─────────────────────────
assert TEST_LOG.count('final evaluation') == 1, ('Unexpected number of final test accesses.')

# ── Display test-access audit ────────────────────────────────────
print('\nFINAL TEST ACCESS AUDIT')
TEST_LOG.show()
print('\nSection 11.1 completed successfully.')
print('Final estimator fitted on training data only.')
print('Held-out test accessed exactly once for final evaluation.')
```

Code 4.13 Held-out scoring (notebook Section 11.2)

```
PRED_TEST11=np.asarray(FINAL_FITTED11.predict(X_TEST11),dtype=float)
OBS_TEST11=np.asarray(y_TEST11,dtype=float)
assert np.isfinite(PRED_TEST11).all() and np.isfinite(OBS_TEST11).all()
assert (PRED_TEST11>0).all(), 'Log RMSE requires positive predictions; investigate, do not clip silently.'
def score11(y,p):
    return {'R²':r2_score(y,p), 'MAE ($)':mean_absolute_error(y,p),
            'RMSE ($)':np.sqrt(mean_squared_error(y,p)),
            'RMSE (log)':np.sqrt(mean_squared_error(np.log(y),np.log(p)))}
TEST_METRICS11=score11(OBS_TEST11,PRED_TEST11)
SCORECARD11=pd.DataFrame([{'Metric':k,'Value':v} for k,v in TEST_METRICS11.items()])
print('Test rows:',len(OBS_TEST11),'| frozen model:',FINAL_MODEL_NAME11)
display(SCORECARD11.round(5))
print('Brief benchmark: R² 0.75–0.85; >0.85 exceeds expectation. Not a model-selection threshold.')
```

Code 4.14 Grouped permutation importance on original features (notebook Section 12.1)

```
# Grouped permutation at ORIGINAL feature level; 3 training-only folds × 2 repeats.
# All preprocessing is refitted within each fold by cloning the frozen estimator.
from sklearn.model_selection import KFold
SEED12=int(SEEDS.get('permutation',99)) if 'SEEDS' in globals() else 99
FOLDS12=KFold(n_splits=3,shuffle=True,random_state=SEED12)
RNG12=np.random.default_rng(SEED12)
IMP_ROWS12=[]
for fold,(tr,va) in enumerate(FOLDS12.split(X12),1):
    est=clone(FINAL_ESTIMATOR11).fit(X12.iloc[tr],y12.iloc[tr])
    xv=X12.iloc[va].copy(); yy=y12.iloc[va].to_numpy()
    base=np.sqrt(mean_squared_error(yy,est.predict(xv)))
    for feature in X12.columns:
        for rep in range(2):
            xp=xv.copy()
            xp[feature]=RNG12.permutation(xp[feature].to_numpy())
            score=np.sqrt(mean_squared_error(yy,est.predict(xp)))
            IMP_ROWS12.append({'Feature':feature,'Fold':fold,'Repeat':rep+1,
                               'Baseline RMSE ($)':base,'RMSE increase ($)':score-base})
IMP_DETAIL12=pd.DataFrame(IMP_ROWS12)
IMP_SUM12=(IMP_DETAIL12.groupby('Feature')['RMSE increase ($)']
          .agg(['mean','std','min','max','count']).reset_index()
          .rename(columns={'mean':'Mean ΔRMSE ($)','std':'SD ΔRMSE ($)',
                           'min':'Minimum ($)','max':'Maximum ($)','count':'Evaluations'})
          .sort_values('Mean ΔRMSE ($)',ascending=False).reset_index(drop=True))
display(IMP_SUM12.head(25))
fig,ax=plt.subplots(figsize=(11,7))
plot12=IMP_SUM12.head(20).iloc[::-1]
ax.barh(plot12['Feature'],plot12['Mean ΔRMSE ($)'],color='#985D7B')
ax.axvline(0,color='#27363F',linewidth=1)
ax.set(xlabel='Increase in validation RMSE after permutation ($)',
       title='Figure 12.1 — Grouped original-feature permutation importance')
plt.tight_layout()
if 'save_fig' in globals(): save_fig(fig,'fig12_1_grouped_permutation_importance',section='12')
plt.show()
print('Importance is training-CV-based; it does not alter the frozen model.')
assert TEST_LOG.count()==TEST_COUNT_AT_12
```

5. Screenshot of the Outputs

The panels below show the key outputs printed by the notebook, in order of the analysis. They are rendered from the notebook's printed results (tables shortened only where noted) so that every figure matches the executed run.

Reading the file

Check	Result
File is the one fingerprinted in 2.3 (SHA-256 equal)	✅
First column is the ID and the last is the target	✅
Rows × columns	1,460 × 81
Predictors (all columns except ID and target)	79
Columns whose other values are all numbers	38 (including ID and target)
Columns that contain the text NA	19 (3 numeric-looking, 16 text)
Columns with blank cells	0
Columns with leading / trailing spaces	0

```
Numeric-looking columns that carry the text NA : PropertyFrontage, ExteriorCladdingArea, BasementYrBlt
Text columns that carry the text NA            : Alley, ExteriorCladdingType, BsmntFinish, BsmntMaintenance, BsmntVisibility, BsmntFinRat1, BsmntFinQual1, E
```

✅ Raw file read as text; NA kept visible. Decision registered.

Output 5.1. Reading the file (Section 3.1): 1,460 × 81, 19 columns with the text NA, no blank or padded cells.

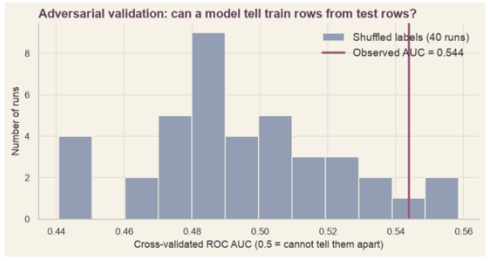
The raw file is read without pandas turning "NA" into a missing value.

The train / test split

Item	Result
Rows: train + test = all rows	1,168 + 292 = 1,460
Test share	20.0%
No PropertyID in both parts	✅
Every PropertyID is in one part	✅
Seed used	SEEDS['split'] = 205
Test-ID fingerprint (SHA-256, first 16)	992f3853012a11f8
Training frame	X_train 1,168 × 79, y_train 1,168
Test rows	inside VAULT only (292 rows)

Adversarial validation: train vs test, predictors only

Item	Result
Observed cross-validated AUC (mean ± SD over 5 folds)	0.544 ± 0.034
Chance-level AUC from shuffled labels (mean)	0.496
Range of shuffled-label AUC (2.5 % to 97.5 %)	0.448 to 0.549
Permutation p-value (40 shuffles)	0.073
Rule: accept the split if p ≥ 0.05	accepted



Labels that never appear in the training rows

Column	Labels only in test	Test rows affected
Heating	Floor	1
AddFeatures	Othr	2

Test-access log

#	Section	Kind	Purpose	Used test target	Time
1	3.4	split diagnostics	adversarial validation and unseen-label check	no	07:03:39

✅ Split made, checked and logged. Section 3 decisions so far: 9

Reading. The reproducible split contains **1,168 training observations (80%)** and **292 held-out observations (20%)**, using random seed 205. No `PropertyID` appears in both partitions.

Adversarial validation was conducted using a random forest trained to distinguish training from test observations using predictors only. The observed five-fold ROC AUC was **0.544 ± 0.034**, compared with a mean AUC of **0.496** under shuffled labels (95% empirical range: 0.448–0.549). The permutation p-value was **0.073**.

Because p = 0.073 exceeds the prespecified 0.05 threshold, the diagnostic did not provide sufficient evidence to reject the null hypothesis of no detectable partition difference. This does **not** establish that the partitions have identical distributions or that future distribution shift is absent.

Two categorical levels occur only in the held-out partition: `Heating = Floor` (one observation) and `AddFeatures = Othr` (two observations). Therefore, the fitted categorical encoders must handle previously unseen levels without failure.

The test predictors were accessed for the documented split diagnostic; test prices were not required for this analysis. Separately, an earlier notebook version evaluated a Gradient Boosting model on the holdout. That historical target-bearing access is disclosed in Sections 11 and 12.

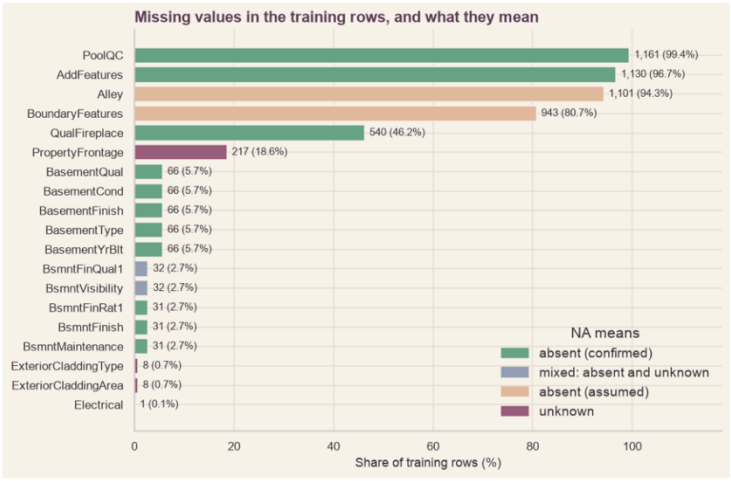
Decision. Retain the original split without redrawing the seed. Use training-fitted preprocessing that handles unseen categories, preserve the split diagnostic in the test-access audit, and treat the final held-out evaluation as supplementary to training-only cross-validation because of historical test-set exposure.

Output 5.2. Train/test split and adversarial validation (Section 3.4).

The split is retained under the pre-set rule because p = 0.073 is above 0.05. This means the diagnostic does not establish a detectable difference at that threshold; it does not prove that the partitions are equivalent. Two labels occur only in test rows, so the fitted preprocessing must handle unseen labels.

Where the training rows are missing a value (19 of 79 predictors)

Column	Type	Missing	Share	Basis	Missing, feature absent	Missing, feature exists	Filled, feature absent	Meaning
PoolQC	text	1,161	99.4%	companion: PoolArea = 0	1161	0	0	absent (confirmed)
AddFeatures	text	1,130	96.7%	companion: AddVal = 0	1130	0	0	absent (confirmed)
Alley	text	1,101	94.3%	no companion column; documentation reading	–	–	–	absent (assumed)
BoundaryFeatures	text	943	80.7%	no companion column; documentation reading	–	–	–	absent (assumed)
QualFireplace	text	540	46.2%	companion: CntFireplaces = 0	540	0	0	absent (confirmed)
PropertyFrontage	number	217	18.6%	every lot has a frontage; there is no "none" label	–	–	–	unknown
BasementCond	text	66	5.7%	companion: garage cars = 0 and garage area = 0	66	0	0	absent (confirmed)
BasementFinish	text	66	5.7%	companion: garage cars = 0 and garage area = 0	66	0	0	absent (confirmed)
BasementQual	text	66	5.7%	companion: garage cars = 0 and garage area = 0	66	0	0	absent (confirmed)
BasementType	text	66	5.7%	companion: garage cars = 0 and garage area = 0	66	0	0	absent (confirmed)
BasementYrBlt	number	66	5.7%	companion: garage cars = 0 and garage area = 0	66	0	0	absent (confirmed)
BsmntFinQual1	text	32	2.7%	companion: basement area = 0	31	1	0	mixed: absent and unknown
BsmntVisibility	text	32	2.7%	companion: basement area = 0	31	1	0	mixed: absent and unknown
BsmntFinRat1	text	31	2.7%	companion: basement area = 0	31	0	0	absent (confirmed)
BsmntFinish	text	31	2.7%	companion: basement area = 0	31	0	0	absent (confirmed)
BsmntMaintenance	text	31	2.7%	companion: basement area = 0	31	0	0	absent (confirmed)
ExteriorCladdingArea	number	8	0.7%	missing on exactly the rows where the veneer type is NA	–	–	–	unknown
ExteriorCladdingType	text	8	0.7%	the column already has its own label None for no veneer, so NA is not "no veneer"; it is missing on the same rows as the area	–	–	–	unknown
Electrical	text	1	0.1%	every house has an electrical system	–	–	–	unknown



After the missing-value rule

Check	Result
Text NA tokens left in any column	0
Text columns where NA became the label None	14 (5,294 cells)
Missing values left, all of them unknown or not applicable	302 cells in 7 columns

What is still missing and goes to the imputation design (3.7)

Column	Missing	Meaning
PropertyFrontage	217	unknown
ExteriorCladdingType	8	unknown
ExteriorCladdingArea	8	unknown
BsmntVisibility	1	unknown (feature exists, value not recorded)
BsmntFinQual1	1	unknown (feature exists, value not recorded)
Electrical	1	unknown
BasementYrBlt	66	not applicable (no garage)

✅ Missing values classified and coded. Section 3 decisions so far: 11

Output 5.3. What NA means in each column (Section 3.5).

Most gaps are structural absence, confirmed by companion columns; only a few columns are genuinely unknown.

Assumed order against price: Spearman, and the direction of the median price across the levels with at least 30 rows

Column	Spearman rank vs log price	CI low	CI high	Levels with ≥ 30 rows	Median price across them	Order supported
ExterQual	+0.69	+0.66	+0.72	3	rising	yes
KitchenQual	+0.69	+0.65	+0.72	4	rising	yes
BsmntFinish	+0.68	+0.64	+0.71	4	rising	yes
BasementFinish	+0.64	+0.60	+0.67	4	rising	yes
QualFireplace	+0.54	+0.50	+0.59	3	rising	yes
HeatingEfficiency	+0.50	+0.45	+0.54	4	rising	yes
BsmntFinRat1	+0.38	+0.33	+0.44	7	mixed	no: mixed order
BasementQual	+0.36	+0.32	+0.41	3	rising	yes
BsmntVisibility	+0.35	+0.30	+0.40	5	rising	yes
BasementCond	+0.35	+0.30	+0.40	2	rising	yes
PavedDrive	+0.29	+0.24	+0.33	2	rising	yes
BsmntMaintenance	+0.27	+0.22	+0.32	4	rising	yes
Functional	+0.13	+0.07	+0.18	1	too few levels	untestable
PoolQC	+0.07	+0.02	+0.11	1	too few levels	untestable
BsmntFinQual1	+0.05	-0.00	+0.11	4	mixed	no: mixed order
Grade	+0.04	-0.02	+0.10	2	rising	no: too weak
ExterCond	+0.03	-0.03	+0.09	2	falling	no: too weak
BoundaryFeatures	-0.19	-0.24	-0.14	4	mixed	no: mixed order
PropertyShape	-0.32	-0.37	-0.26	3	falling	yes

Rule: an order is "supported" if the median price across the levels with ≥ 30 rows moves in ONE direction (rising or falling: a falling price is still a valid order) and |Spearman| is at least 0.10; "no: mixed order" if it zig-zags, "no: too weak" if |Spearman| is below 0.10, "untestable" if fewer than two levels have 30 rows.
Not supported: ['BsmntFinRat1', 'BsmntFinQual1', 'Grade', 'ExterCond', 'BoundaryFeatures']. Untestable: ['Functional', 'PoolQC'].

Output 5.4. Ordinal orders tested against price (Section 3.9).

12 orders are supported, 5 are not and 2 cannot be tested; no column is dropped.

Q0. Tail guard (ridge probe, log price) (15 paired folds; lower RMSE is better)

Variant	RMSE (log)	SD over folds	Change vs baseline	CI low	CI high	Beats the rule
No guard (baseline)	0.1483	0.0304	+0.00000	+0.00000	+0.00000	baseline
Clip at ±5 SD	0.1373	0.0193	-0.01092	-0.01799	-0.00491	yes
Clip at ±4 SD	0.1339	0.0177	-0.01435	-0.02220	-0.00759	yes
Clip at ±3 SD	0.1304	0.0159	-0.01785	-0.02652	-0.01041	yes
Clip at ±2.5 SD	0.1293	0.0150	-0.01901	-0.02795	-0.01132	yes
Clip at ±2 SD	0.1290	0.0145	-0.01925	-0.02845	-0.01151	yes

Q1. Ordinal columns (ridge probe, log price) (15 paired folds; lower RMSE is better)

Variant	RMSE (log)	SD over folds	Change vs baseline	CI low	CI high	Beats the rule
Codes in the declared order (baseline)	0.1304	0.0159	+0.00000	+0.00000	+0.00000	baseline
One-hot, order ignored	0.1297	0.0140	-0.00072	-0.00217	+0.00046	no
Codes plus one-hot	0.1316	0.0159	+0.00116	+0.00048	+0.00188	no
Codes, the 5 unsupported orders as one-hot	0.1305	0.0159	+0.00010	-0.00020	+0.00043	no

Q2. Nominal columns (ridge probe, log price) (15 paired folds; lower RMSE is better)

Variant	RMSE (log)	SD over folds	Change vs baseline	CI low	CI high	Beats the rule
One-hot, every level (baseline)	0.1304	0.0159	+0.00000	+0.00000	+0.00000	baseline
Levels under 10 rows pooled	0.1307	0.0160	+0.00027	+0.00005	+0.00051	no
Levels under 30 rows pooled	0.1314	0.0162	+0.00097	+0.00054	+0.00138	no

Q3. Month sold (ridge probe, log price) (15 paired folds; lower RMSE is better)

Variant	RMSE (log)	SD over folds	Change vs baseline	CI low	CI high	Beats the rule
One-hot, 12 months (baseline)	0.1304	0.0159	+0.00000	+0.00000	+0.00000	baseline
Cyclic: sine and cosine	0.1303	0.0159	-0.00011	-0.00053	+0.00034	no

Q4. Scaling (ridge probe, log price) (15 paired folds; lower RMSE is better)

Variant	RMSE (log)	SD over folds	Change vs baseline	CI low	CI high	Beats the rule
Standardised numbers (baseline)	0.1304	0.0159	+0.00000	+0.00000	+0.00000	baseline
Raw numbers	0.1487	0.0316	+0.01831	+0.01036	+0.02777	no

Q5. Gradient-boosting probe (untuned, log price) (15 paired folds; lower RMSE is better)

Variant	RMSE (log)	SD over folds	Change vs baseline	CI low	CI high	Beats the rule
Ordinal codes, one-hot nominal (baseline)	0.1339	0.0088	+0.00000	+0.00000	+0.00000	baseline
Ordinal codes, native categories for nominal	0.1312	0.0074	-0.00271	-0.00472	-0.00089	yes

The final linear design on 60 training rows with problems created on purpose

Scenario	All predictions finite	Median |change in log price| (the 30 changed rows)	Largest |change|
Control: nothing changed	✅	0.0000	0.0000
A label never seen in a nominal column (Condition2) in 30 rows	✅	0.0224	0.0224
A label never seen in an ordinal column (ExterQual) in 30 rows	✅	0.0000	0.0219
A label never seen in a binary column (CentralAir) in 30 rows	✅	0.0000	0.0595
Missing labels in an ordinal and a nominal column (BsmntVisibility, SaleType) in 30 rows	✅	0.0000	0.0721
Missing numbers (GrLivArea, PropertySize) in 30 rows	✅	0.0403	0.2762

```
Final encoding: {'tail': 3, 'ordinal': 'codes', 'month': 'onehot', 'scale': True}.  Tree family: native categories = True.
Columns after encoding: 246  (nom 190, num 34, ord_codes 19, bin 3).
✅ Encoding designed; fit is reproducible; no missing value reaches the model.

✅ Section 3 decisions so far: 24
```

Output 5.5. Tail guard and encoding alternatives (Section 3.10).

Clipping materially improves the linear-model probe, with ±3 SD retained under the disclosed amended rule. The tested linear-family encoding alternatives do not meet the adoption criterion. The separate HistGradientBoosting experiment favours native categorical handling, but that pipeline is not used for the Section 10 candidate comparison.

```
Engineered features, missing values: none
```

Table 9.7b: engineered feature groups (15 paired folds, ridge probe with tail guard)

Variant	RMSE (log)	RMSE ($)	MAE ($)	Change in log RMSE	CI low	CI high	Beats the rule
Baseline (no engineered features)	0.1302	$30,992	$16,200	+0.00000	+0.00000	+0.00000	baseline
+ Age features	0.1303	$31,166	$16,207	+0.00003	-0.00017	+0.00026	no
+ Total living area	0.1295	$30,615	$16,109	-0.00077	-0.00094	-0.00060	no
+ Outdoor area	0.1303	$31,087	$16,209	+0.00010	-0.00000	+0.00024	no
+ Total bathrooms	0.1303	$31,027	$16,208	+0.00010	+0.00004	+0.00017	no
+ Quality × area	0.1298	$30,577	$16,078	-0.00042	-0.00069	-0.00015	no
+ All engineered	0.1294	$30,610	$16,024	-0.00083	-0.00124	-0.00039	no

```
Adopted engineered features: none
```

Output 5.6. PCA versus accuracy (Section 9.7).

Only 23 components match the no-PCA error; fewer cost accuracy.

	Model	RMSE_mean	RMSE_sd	MAE_mean	R2_mean	log_RMSE_mean
0	Small OLS	40774.203	5191.971	26713.188	0.745	0.217
1	Dummy mean	81292.598	7795.155	58466.097	-0.006	0.413
2	Dummy median	83131.197	8726.494	56522.966	-0.051	0.405

```
Small OLS features: ['GrLivArea', 'OverallQual', 'YearBuilt', 'PropertySize', 'BasementCars']
```

	Model	RMSE_mean	RMSE_sd	MAE_mean	R2_mean	log_RMSE_mean
0	Gradient Boosting	29779.426	5704.554	16705.126	0.862	0.132
1	ElasticNet	30627.745	8970.728	15651.093	0.846	0.129
2	Lasso	30660.821	8815.875	15690.513	0.847	0.129
3	Ridge	30992.301	8664.811	16199.888	0.844	0.130
4	ExtraTrees	31419.798	5611.317	17939.489	0.847	0.145
5	Random Forest	32545.347	5841.438	18429.112	0.836	0.147
6	Linear Regression	37143.235	17229.419	17779.883	0.762	0.147
7	Decision Tree	41543.890	6967.873	26829.593	0.734	0.201
8	KNN	51561.311	7871.676	31489.054	0.594	0.237
9	SVR	64839.408	9085.315	40082.603	0.359	0.288

Output 5.7. Baselines and ten candidate algorithms (Section 10.2–10.3).

Gradient Boosting leads on RMSE, penalised linear models on MAE; seven of ten beat the baseline.

	Model	RMSE_mean	RMSE_sd	MAE_mean	R2_mean	log_RMSE_mean
0	Lasso	30376.865	9771.146	15726.823	0.852	0.129
1	ElasticNet	30502.665	10166.589	15871.260	0.850	0.130
2	Gradient Boosting	30649.103	5895.373	17084.545	0.855	0.133

	Model	Fold	Best parameters	Inner best RMSE ($)
0	Gradient Boosting	1	{'regressor__model__learning_rate': 0.03580868...	29721.709914
1	Gradient Boosting	2	{'regressor__model__learning_rate': 0.08788217...	35183.646377
2	Gradient Boosting	3	{'regressor__model__learning_rate': 0.12709795...	28397.381733
3	Gradient Boosting	4	{'regressor__model__learning_rate': 0.12011000...	31892.061595
4	Gradient Boosting	5	{'regressor__model__learning_rate': 0.13405556...	33522.338998
5	ElasticNet	1	{'regressor__model__alpha': 0.0019596814681528...	29569.540513
6	ElasticNet	2	{'regressor__model__alpha': 0.0010627607992094...	35014.995338
7	ElasticNet	3	{'regressor__model__alpha': 0.0022072209737041...	27104.676180
8	ElasticNet	4	{'regressor__model__alpha': 0.0050132190301292...	34264.855664
9	ElasticNet	5	{'regressor__model__alpha': 0.0214643246193002...	36977.210234
10	Lasso	1	{'regressor__model__alpha': 0.0009196430215927...	29619.668933
11	Lasso	2	{'regressor__model__alpha': 0.000718487244344908}	35027.009683
12	Lasso	3	{'regressor__model__alpha': 0.0006981720930708...	26664.452211
13	Lasso	4	{'regressor__model__alpha': 0.0006349988758323...	34637.837958
14	Lasso	5	{'regressor__model__alpha': 0.005462879602646594}	36998.779382

	Model	RMSE_mean	RMSE_sd	MAE_mean	R2_mean	log_RMSE_mean
0	Lasso + Gradient Boosting (average)	28963.845	6670.811	15401.465	0.867	0.125
1	Gradient Boosting	29779.426	5704.554	16705.126	0.862	0.132
2	Lasso	30660.821	8815.875	15690.513	0.847	0.129

```
Exploratory only: compare a tuned ensemble under nested CV before adopting it.
```

	Model	RMSE_mean	RMSE_sd	MAE_mean	R2_mean	log_RMSE_mean
0	Lasso + Gradient Boosting (average)	28870.618	7323.912	15399.643	0.869	0.125
1	Lasso	30376.865	9771.146	15726.823	0.852	0.129
2	Gradient Boosting	30649.103	5895.373	17084.545	0.855	0.133

Table 10.6b: average minus member, per metric (negative favours the average; 5 dependent folds, exploratory)

Metric	Average vs	Mean difference	Interval low	Interval high	Folds where the average is better	of
RMSE ($)	Lasso	-1,506.2471	-4,917.2019	+1,904.7078	3	5
RMSE ($)	Gradient Boosting	-1,778.4846	-5,261.8048	+1,704.8355	4	5
MAE ($)	Lasso	-327.1799	-871.7732	+217.4134	3	5
MAE ($)	Gradient Boosting	-1,684.9022	-2,276.4036	-1,093.4008	5	5
RMSE (log)	Lasso	-0.0044	-0.0082	-0.0006	5	5
RMSE (log)	Gradient Boosting	-0.0079	-0.0150	-0.0009	5	5

```
vs Lasso: RMSE better in 3 of 5 folds; mean MAE difference −327; mean log-RMSE difference −0.0044
vs Gradient Boosting: RMSE better in 4 of 5 folds; mean MAE difference −1,685; mean log-RMSE difference −0.0079
Adopt the average as the final model: False
```

Output 5.8. Nested cross-validation and ensemble check (Section 10.4–10.6).

The exploratory paired comparisons do not establish a clear RMSE advantage among Lasso, ElasticNet and Gradient Boosting. Lasso is selected using the documented simplicity and error criteria. The average ensemble has lower mean RMSE but does not meet the adoption rule.

```
Frozen parameters (training-only search): {'regressor__model__alpha': np.float64(0.0013125743984880442)} | inner-CV RMSE $33,252
FROZEN MODEL CONFIGURATION
Model: Lasso
Configuration SHA256: 661bd87cedee8f1aba75d58ece2c1b4d83c943c8f26e52a4bcbbc9cb7b4ecdf2
Explicit parameter overrides: {'regressor__model__alpha': np.float64(0.0013125743984880442)}

Lasso hyperparameters:
  alpha: 0.0013125743984880442
  copy_X: True
  fit_intercept: True
  max_iter: 200000
  positive: False
  precompute: False
  random_state: 2026
  selection: random
  tol: 0.0001
  warm_start: False

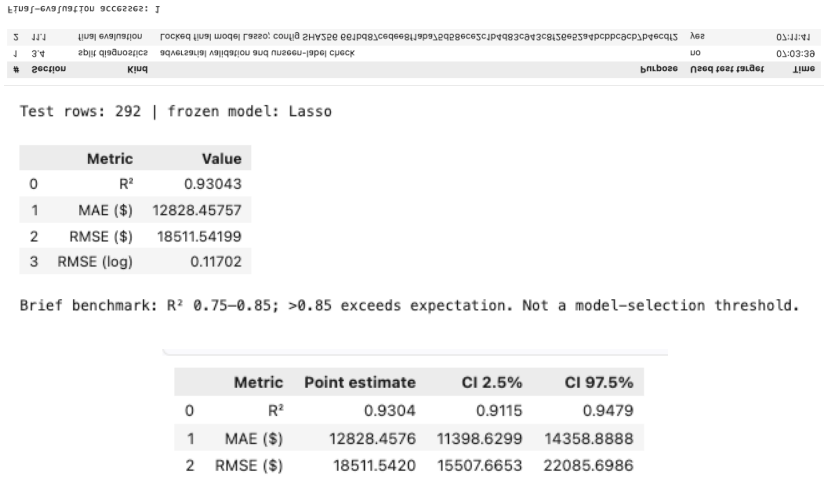
FINAL TEST ACCESS AUDIT
```

#	Section	Kind	Purpose	Used test target	Time
1	3.4	split diagnostics	adversarial validation and unseen-label check	no	07:03:39
2	11.1	final evaluation	Locked final model Lasso; config SHA256 661bd87cedee8f1aba75d58ece2c1b4d83c943c8f26e52a4bcbbc9cb7b4ecdf2	yes	07:11:41

```
Section 11.1 completed successfully.
Final estimator fitted on training data only.
Held-out test accessed exactly once for final evaluation.
```

Output 5.9. Frozen model and test-access audit (Section 11.1).

The model is locked and fingerprinted before the held-out rows are opened, and the log shows one target-bearing access.



Test rows: 292 | frozen model: Lasso

	Metric	Value
0	R^2	0.93043
1	MAE ($)	12828.45757
2	RMSE ($)	18511.54199
3	RMSE (log)	0.11702

Brief benchmark: R^2 0.75–0.85; >0.85 exceeds expectation. Not a model-selection threshold.

	Metric	Point estimate	CI 2.5%	CI 97.5%
0	R^2	0.9304	0.9115	0.9479
1	MAE ($)	12828.4576	11398.6299	14358.8888
2	RMSE ($)	18511.5420	15507.6653	22085.6986

Output 5.10. Held-out metrics with bootstrap intervals (Section 11.2–11.3).

R^2 0.930, MAE $12,828, RMSE $18,512 on 292 sales.

	Metric	CV mean	CV fold SD	Test	Test – CV mean
0	R^2	0.8467	0.0822	0.9304	0.0837
1	MAE ($)	15817.2791	1421.2687	12828.4576	-2988.8215
2	RMSE ($)	30701.0194	8674.9143	18511.5420	-12189.4774
3	RMSE (log)	0.1293	0.0153	0.1170	-0.0123

Post-hoc diagnostic: no model change authorised. Repeated folds are dependent.

Output 5.11. Cross-validation versus held-out test (Section 11.4).

The held-out result is better than cross-validation by about one fold SD in R^2.

	Feature	Mean ΔRMSE ($)	SD ΔRMSE ($)	Minimum ($)	Maximum ($)	Evaluations
0	GrLivArea	14871.213620	7899.975608	3510.929590	21185.830300	6
1	OverallQual	8977.545015	2701.431393	5490.365357	12362.765193	6
2	BsmntSqFtage	2356.078855	3097.907136	-1918.540391	5557.907446	6
3	YearBuilt	2346.182178	944.544904	1008.383327	3396.795320	6
4	OverallCond	1779.594389	1408.849239	347.878556	3557.960196	6
5	BasementCars	1703.743913	1197.020446	-61.019819	3320.543570	6
6	Neighborhood	1593.535998	781.238765	623.105138	2518.881193	6
7	1stFlrSF	959.370048	1079.280316	0.000000	2949.280708	6
8	Condition1	883.443079	605.511727	139.289092	1569.391041	6
9	BsmntFinSty1	771.555712	1208.042318	-1128.377899	1853.187358	6
10	ScreenPorch	610.555513	701.949786	-258.054132	1741.537945	6
11	KitchenQual	593.365801	725.231040	-335.590837	1548.332373	6
12	Functional	575.368954	345.858176	265.089261	1249.842224	6
13	PropertyClass	509.974934	379.915135	22.141815	1038.383246	6
14	PropertyZone	416.139872	476.998964	-80.926178	961.893570	6
15	PropertyFooting	371.803550	311.469507	-23.973895	815.019039	6
16	HeatingEfficiency	368.348196	586.221744	-523.336526	1176.459493	6
17	Roof1Material	319.632601	494.714038	-61.822841	1282.766708	6
18	QualFireplace	313.461698	259.614562	50.984203	650.276730	6
19	BasementSqFootage	272.201686	359.222672	-58.450443	718.069799	6
20	BasementFinish	268.414683	422.074542	0.000000	919.649665	6
21	YearRemodAdd	241.732940	558.030240	-585.240884	872.674738	6
22	EnclosedPorch	206.021249	409.147155	-87.266466	1009.663521	6
23	BsmntVisibility	201.711228	855.782314	-1036.787118	1131.731877	6
24	PropertySize	179.191379	1996.803808	-2059.880764	2752.414152	6

Output 5.12. Grouped permutation importance (Section 12.1).

GrLivArea and OverallQual dominate.

```
Calibration n=234; conformal radius=$20,597.17; nominal=80%
Observed overall coverage=82.9%; Wilson 95% CI=[78.1%,86.8%]
Mean interval width=$41,194.33

Post-hoc interval coverage by Observed price tier
```

Observed price tier	n	coverage	width	Wilson lower	Wilson upper
Q1	73	0.890	41194.331	0.798	0.943
Q2	73	0.877	41194.331	0.782	0.934
Q3	73	0.877	41194.331	0.782	0.934
Q4	73	0.671	41194.331	0.557	0.768

```
Post-hoc interval coverage by Neighborhood
```

Neighborhood	n	coverage	width	Wilson lower	Wilson upper
Blmngtn	6	1.000	41194.331	0.610	1.000
BrkSide	10	0.700	41194.331	0.397	0.892
ClearCr	7	0.286	41194.331	0.082	0.641
CollgCr	35	0.857	41194.331	0.706	0.937
Crawfor	12	0.583	41194.331	0.320	0.807
Edwards	21	0.905	41194.331	0.711	0.973
IDOTRR	6	0.667	41194.331	0.300	0.903
Mitchel	10	0.900	41194.331	0.596	0.982
NAmes	45	0.933	41194.331	0.821	0.977
NWAmes	15	0.867	41194.331	0.621	0.963
NoRidge	9	0.667	41194.331	0.354	0.879
NridgHt	13	0.769	41194.331	0.497	0.918
OldTown	26	0.923	41194.331	0.759	0.979
SWISU	7	0.714	41194.331	0.359	0.918
Sawyer	9	0.778	41194.331	0.453	0.937
SawyerW	13	0.923	41194.331	0.667	0.986
Somerst	21	0.762	41194.331	0.549	0.894
StoneBr	5	0.800	41194.331	0.376	0.964
Timber	8	0.625	41194.331	0.306	0.863

```
Post-hoc interval coverage by SaleCondn
```

SaleCondn	n	coverage	width	Wilson lower	Wilson upper
Abnorml	23	0.870	41194.331	0.679	0.955
Normal	233	0.845	41194.331	0.794	0.886
Partial	30	0.700	41194.331	0.521	0.833

```
Do not tune conformal alpha, calibration split, or model using these test outcomes.
```

Output 5.13. Coverage of the 80% prediction intervals (Section 11.6).

Overall coverage 82.9%, but 67.1% in the top price quarter and 70.0% for Partial sales.

6. Report on EDA

All exploratory analysis uses the **1,168 training rows only**; the 292 held-out rows were not used. No column or row was removed in this section, as the brief requires. Flagged values are review shortlists, not deletion lists. Figures were redrawn from the training rows and reproduce the notebook's reported statistics (for example price skewness +1.95, Spearman ρ of +0.812 for OverallQual).

6.1 Data collection and quality checks

The data needed careful checking before analysis. Each problem was measured, treated in a stated way and carried through to the model.

ISSUE	WHAT WE FOUND	WHAT WE DID	WHY IT MATTERS
NA IS NOT "MISSING"	19 columns contain the text NA. In most it means the feature is absent (no pool, alley, garage, fireplace).	Read cells as text, then interpreted NA using companion variables where available and documented assumptions where no companion exists.	Pandas would silently turn absent features into missing values and lose the information.
NAMES DIFFER FROM THE BRIEF	7 of 79 positions differ: LotOrientation→Orientation, BsmtBath1→BsmtFullBath, BsmtBath2→BsmtHalfBath, Kitchen→KitchenUpLev, SquareFootage→BasementSqFootage, BasementSqFootage→BasementCond, YrSold→SaleYr.	Used the CSV names; kept a name map so each brief name can be traced.	Avoids mapping a description to the wrong column.
MISLEADING NAMES	Seven "Basement…" columns (BasementType to BasementCond) hold garage data. Roof1Material and Roof2Material hold exterior covering, not roof material.	Recorded the corrected meanings in the metadata sheet; kept the names.	BasementCars is garage capacity, not a basement feature.
INCONSISTENT SPELLING	Roof2Material spells three labels differently from Roof1Material (Brk Cmn, CmentBd, Wd Shng).	Fixed relabelling by lookup (nothing is fitted).	The two columns now share one label set.
CODES STORED AS NUMBERS	PropertyClass and SaleMon are labels stored as numbers.	Stored as text and treated as nominal.	Averages of building-class codes mean nothing.
GENUINE GAPS	After structural-absence rules, 302 entries remain missing or not applicable across seven columns, including 217 unknown frontage values and 66 garage years that are not applicable.	Applied branch-specific treatment within folds: numeric medians, categorical modes and a garage-year transformer distinguishing absence from an unknown year.	Gap rows sell for a median $13,000 more (CI $2,010 to $22,221), yet knowing which rows were filled adds nothing to prediction.
INTEGRITY	All 12 must-hold rules pass, including area identities, construction before sale, remodel year not preceding construction, valid ratings and nonnegative areas/counts. Eleven rows receive plausibility flags. Remodel after sale is a plausibility flag, not a failed must-hold rule.	Nothing changed or removed; flagged rows are judged by cross-validation.	A remodel recorded after the sale (ID 524) is unusual but possible.
DUPLICATES AND IDS	PropertyID unique; no duplicate rows ignoring the ID; target numeric and complete.	No row removed.	Nothing can leak across the split as a duplicate.
TRAIN/TEST SIMILARITY	Adversarial AUC 0.544 ± 0.034 against 0.496 for shuffled labels (p = 0.073). Two labels occur only in test rows (Heating = Floor, AddFeatures = Othr).	Retained the split under the stated rule; fitted encoders use their specified unseen-category fallbacks.	Guards against a split that differs by chance in a detectable way.

6.2 Missing values: absent or unknown

Of the 79 predictors, 19 have a missing value in the training rows. Eleven are missing **only where the feature does not exist** (confirmed by a companion column such as PoolArea = 0 or CntFireplaces = 0), two are mixed (one unknown row each), two rest on an assumption (Alley, BoundaryFeatures, which have no companion column) and four are unknown (PropertyFrontage, ExteriorCladdingType, ExteriorCladdingArea, Electrical). The five garage columns are missing on exactly the 66 rows where garage cars and garage area are both zero.

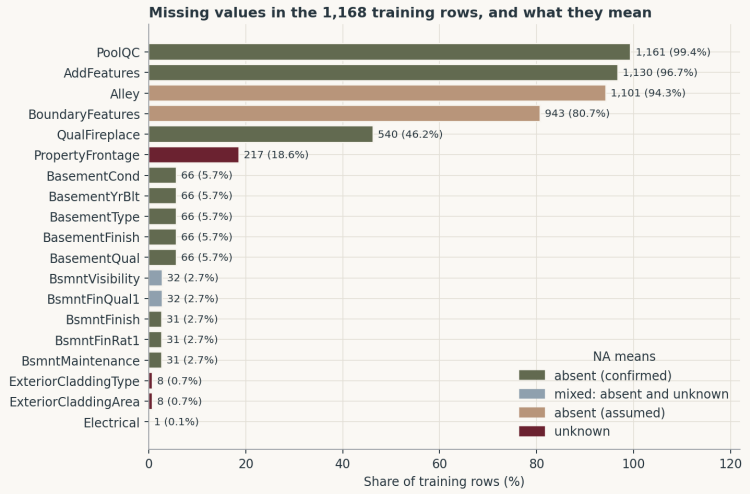


Figure 6.1. Missing values in the training rows and what each means.

After the fixed rule, no text NA is left; 14 text columns carry the label None (5,294 cells) and 302 cells in 7 columns remain missing, all unknown or not applicable.

> **So what:** Treating absence as its own label keeps real information. Properties with a fireplace, garage or basement sell for about 52%, 75% and 60% more than those without, and properties with alley access about 21% less (associations, not causes; Section 6.6).

6.3 The target variable

PropPrice is strongly right-skewed (skewness +1.95, excess kurtosis +6.76). The log removes most of it (+0.18, kurtosis +0.75). Residual spread in a simple regression grows with fitted price on the raw scale (Spearman |residual| vs fitted +0.23) but not on the log scale (−0.06), and on the final design the log model also gives the lowest dollar RMSE ($31,027), MAE ($15,874) and highest R^2 (0.840) of three scales tried (none, log, Yeo-Johnson). The arithmetic mean price ($181,554) is 8.8% above the geometric mean ($166,876).

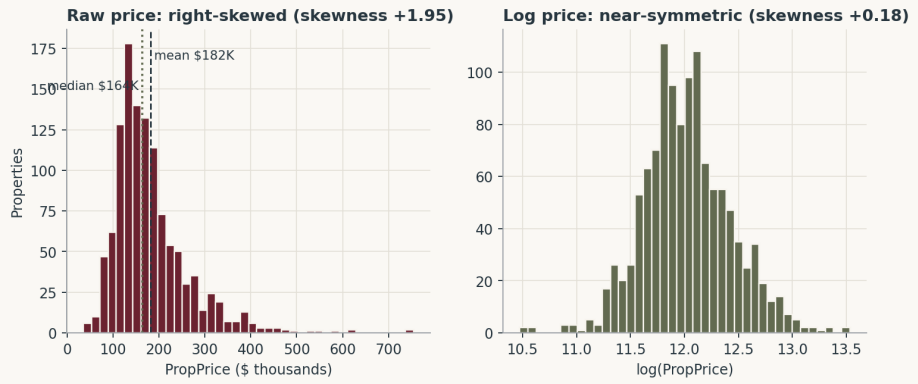


Figure 6.2. Distribution of PropPrice on the raw and log scales (training rows).

Because the model predicts log price, predictions are converted back with exp(). Duan's smearing correction (mean factor 1.0059) was tested and not adopted: it changed RMSE by −$60 (CI −$154 to +$34) and MAE by +$22 (CI −$20 to +$70).

6.4 Numeric predictors

The 34 numeric predictors are 19 sizes, 2 scores (1–10), 4 years and 9 counts. Thirteen columns are reliably skewed (bootstrap interval beyond ±1), among them PropertySize (+12.16), PoolArea (+13.24), AddVal (+23.85) and the porch areas. Thirteen are mostly zeros because the feature is absent for most properties: PoolArea is 99.4% zeros (7 pools), LowQualFinSF 98.3%, 3SsnPorch 98.2% and AddVal 96.7%. For those, means mislead, so the median of the non-zero values is reported alongside (a median pool is 555 sq ft).

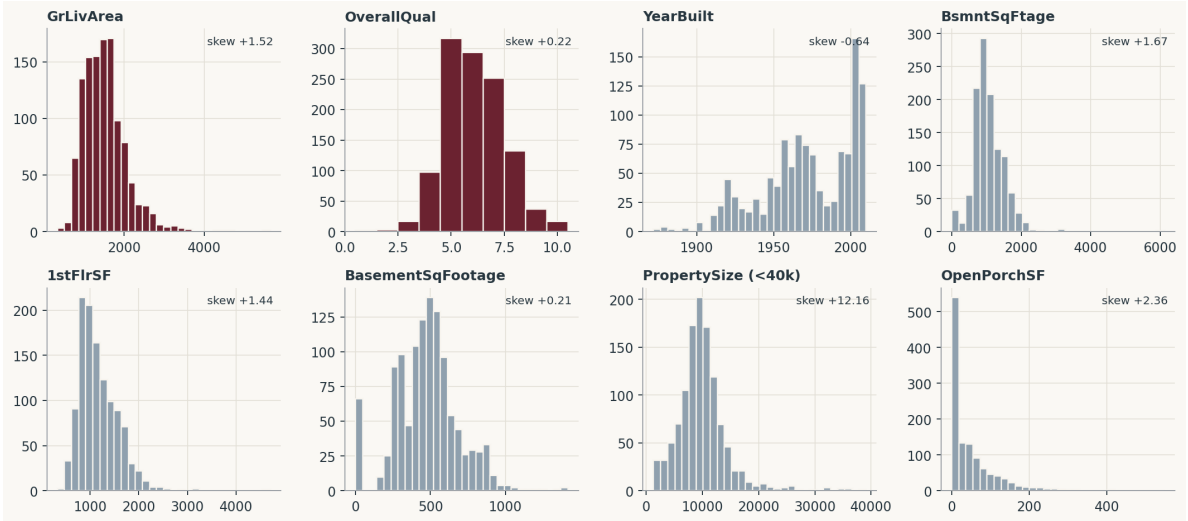


Figure 6.3. Distributions of eight key numeric predictors (cleaned training rows).

PREDICTOR	MEDIAN	MEAN	MIN – MAX	SKEWNESS (95% CI)
GRLIVAREA (SQ FT)	1,467	1,520	334 – 5,642	+1.52 (+0.98 to +2.08)
PROPERTYSIZE (SQ FT)	9,600	10,679	1,300 – 215,245	+12.16 (+4.23 to +14.83)
OVERALLQUAL (1–10)	6	6.10	1 – 10	+0.22 (+0.06 to +0.38)
YEARBUILT	1,973	1,972	1,872 – 2,010	−0.64 (−0.74 to −0.53)
BSMNTSQFTAGE (SQ FT)	995	1,065	0 – 6,110	+1.67 (+0.34 to +3.20)
PROPERTYFRONTAGE (FT; 951 RECORDED)	70	70.8	21 – 313	+2.34 (+0.61 to +3.46)

6.5 Outliers and influential points

Thirty-three numeric columns were screened with the IQR rule and the modified z-score, 172 properties (14.7%) are jointly unusual by robust Mahalanobis distance, and 57 exceed the Cook-distance cut-off of 4/n. These flags are shortlists: the chi-square reference assumes a roughly normal cloud, and these columns are skewed and zero-heavy. The most informative cases are the four properties above 4,000 sq ft.

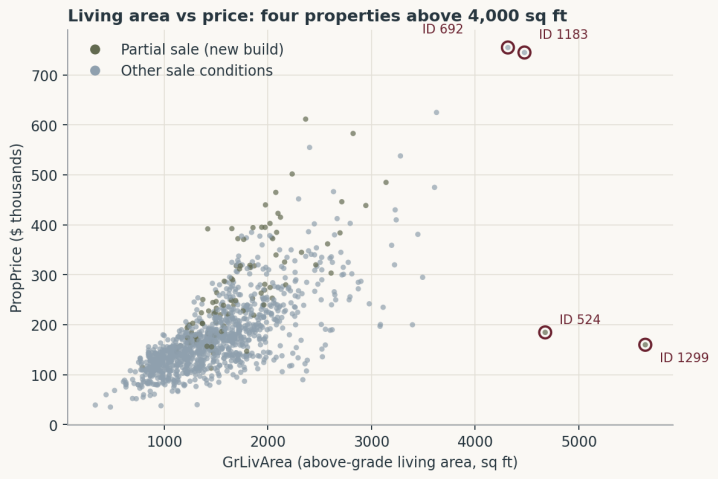


Figure 6.4. Living area against price; the four properties above 4,000 sq ft are circled.

ID	GRLIVAREA	PRICE	SALE CONDITION	NEIGHBORHOOD	$ PER SQ FT	PEER MEDIAN	COOK D
524	4,676	$184,750	Partial (new)	Edwards	$39.5	$108.4	0.411
692	4,316	$755,000	Normal	NoRidge	$174.9	$169.4	0.0028
1183	4,476	$745,000	Abnormal	NoRidge	$166.4	$173.6	0.0021
1299	5,642	$160,000	Partial (new)	Edwards	$28.4	$114.0	4.847

IDs 692 and 1183 are large but have prices per square foot close to their peer medians, with ratios of 1.03 and 0.96. IDs 524 and 1299 have ratios of 0.36 and 0.25, each based on only two peers. They are candidate atypical sales rather than confirmed errors. With the ±3 SD guard, excluding them increases mean RMSE by $337, with fold differences ranging from −$3,755 to +$5,643 and repeat means of +$682, −$255 and +$584. Without the guard, exclusion increases mean RMSE by $16,942, although that experiment is highly unstable because of extreme extrapolation. All rows are retained because exclusion does not provide a consistent guarded RMSE improvement.

6.6 What is associated with price

Numeric predictors

Twenty-eight of the 33 quantitative predictors are associated with log price after Benjamini–Hochberg adjustment. The strongest are OverallQual (Spearman ρ = +0.812, 95% interval +0.787 to +0.834), GrLivArea (+0.730), BasementCars (+0.698), YearBuilt (+0.660), BasementSqFootage (+0.656) and Bath1 (+0.641). Mutual information also ranks OverallQual and GrLivArea highly. Negative rank associations include EnclosedPorch (−0.217), KitchenUpLev (−0.163) and OverallCond (−0.096). OverallCond's Pearson correlation is −0.006; these weak coefficients do not establish the shape or monotonicity of its relationship. PropertySize has Spearman ρ = +0.471 and Pearson r = +0.273, indicating that its rank and linear associations differ.

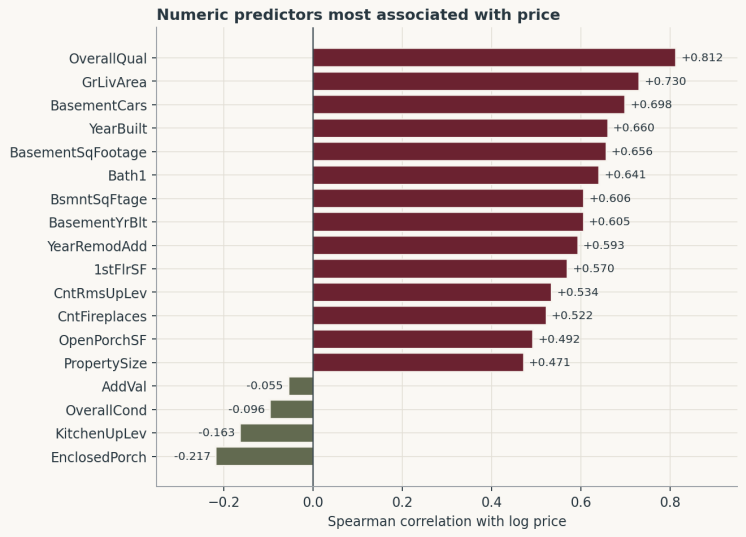


Figure 6.5. Spearman correlation of numeric predictors with log price (strongest 14 and weakest 4).

Ordinal and nominal predictors

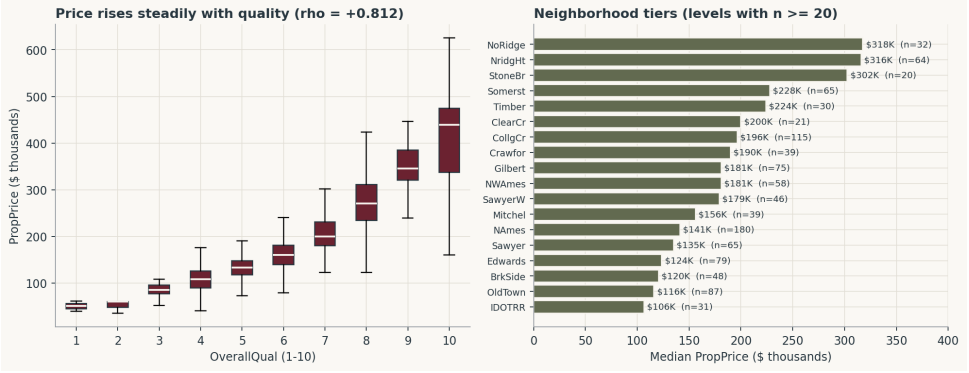


Figure 6.6. Price by overall quality (left) and median price by neighborhood (right).

Several quality and finish grades have strong rank associations with log price: ExterQual ρ = +0.691, KitchenQual +0.687, BsmntFinish +0.675, BasementFinish +0.637, QualFireplace +0.543 and HeatingEfficiency +0.501. The Section 3 order check supports 12 of the 19 declared ordinal scales, including PropertyShape with a decreasing price relationship. Five scales are unsupported: BsmntFinRat1, BsmntFinQual1 and BoundaryFeatures have mixed ordering, while Grade and ExterCond have weak associations. Functional and PoolQC cannot be tested adequately. OverallCond is a numeric score in the project metadata, rather than one of these 19 ordinal columns. Among the nominal and binary predictors, Neighborhood has the strongest group effect (rank ε^2 = 0.575), followed by garage type (0.370), PropertyClass (0.361) and PropertyFooting (0.328). No material sale-month association is detected in this sample (ε^2 approximately zero, p = 0.66).

What “None” carries

Differences in mean log price are: fireplace +0.416 (95% CI +0.377 to +0.455, about 52% higher), garage +0.561 (+0.478 to +0.644, about 75%), basement +0.471 (+0.377 to +0.575, about 60%) and alley access −0.236 (−0.328 to −0.151, about 21% lower). A pool shows +0.403 but rests on only 7 pools (Welch p = 0.088). Absence of a garage or basement goes with older, smaller houses, so none of this is read as the effect of the feature itself.

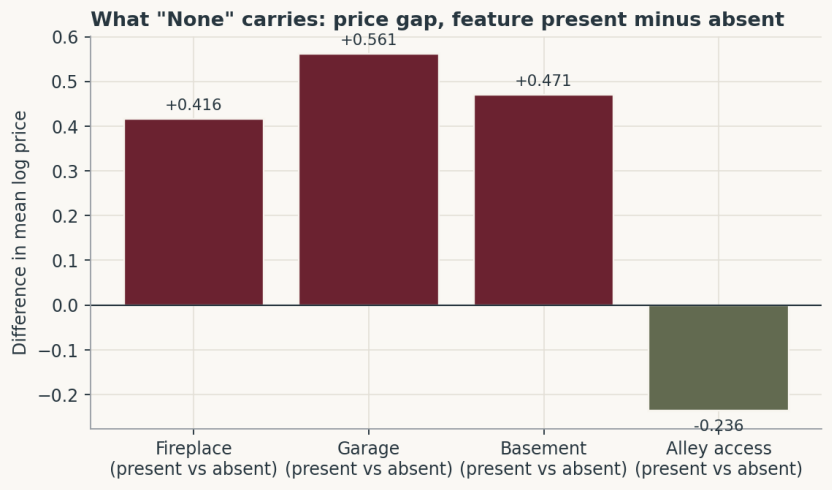


Figure 6.7. Difference in mean log price when a feature is present rather than absent.

6.7 Multicollinearity and redundancy

Redundancy is structural, not accidental. Four numeric pairs exceed |ρ| = 0.80: YearBuilt with the garage year (+0.894), garage capacity with garage area (+0.854), basement area with first-floor area (+0.828) and living area with room count (+0.824). Two exact accounting identities hold in all 1,168 rows (above-grade living area = first floor + second floor + low-quality area; basement total = finished 1 + finished 2 + unfinished), and PropertyClass 90 duplicates BldgType Duplex. In a reference design with 90 encoded terms this makes the matrix rank 87, so there are three exact dependencies and ten infinite variance inflation factors; 18 further terms exceed VIF 10 (PropertyClass 20 at 160.7, PropertyClass 60 at 118.0, BldgType TwnhsE 49.0, PropertyZone RL 37.4, YearBuilt 11.6), while OverallQual (4.1) is not a concern.

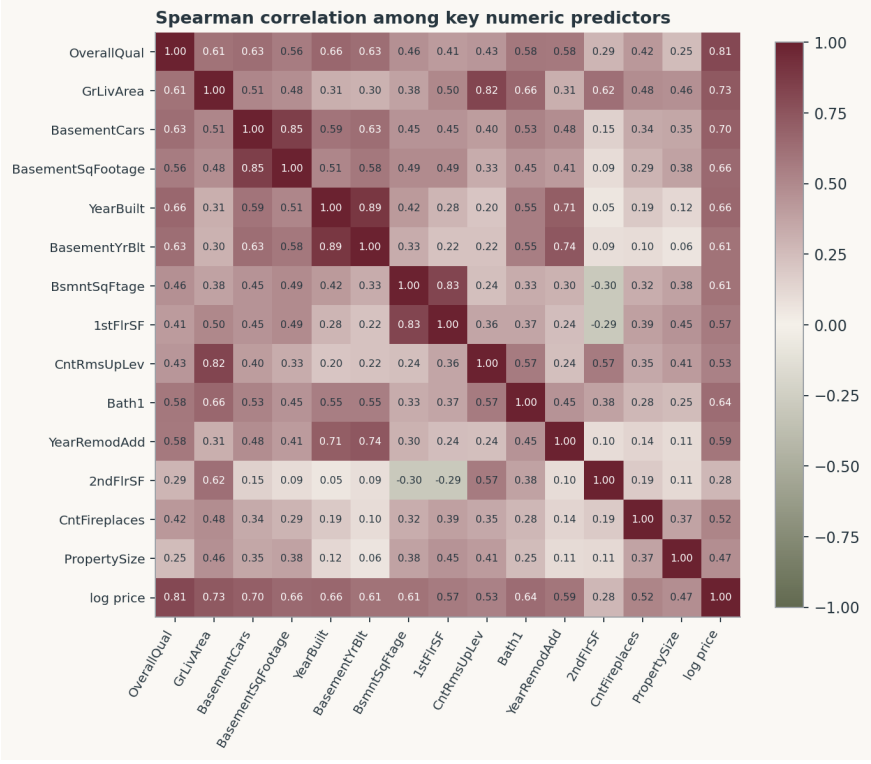


Figure 6.8. Spearman correlations among key numeric predictors and log price.

Five nominal pairs exceed a corrected Cramér's V of 0.50: PropertyClass with BldgType (0.883) and PropertyStyle (0.841), the two exterior coverings (0.770), garage type with garage finish (0.683) and PropertyZone with Neighborhood (0.623). **No predictor was dropped**, in line with the brief; the penalty in Ridge, Lasso and ElasticNet manages the redundancy, and importance in Section 8.3 is read by original feature rather than by single coefficient.

6.8 Preprocessing and feature-engineering experiments

Every preprocessing alternative was tested against the chosen design on 15 paired folds. Figure 6.9 shows the tail guard, which is the one change that clearly helps; Figure 6.10 shows PCA; Figure 6.11 shows that every other alternative fails to beat the chosen design.

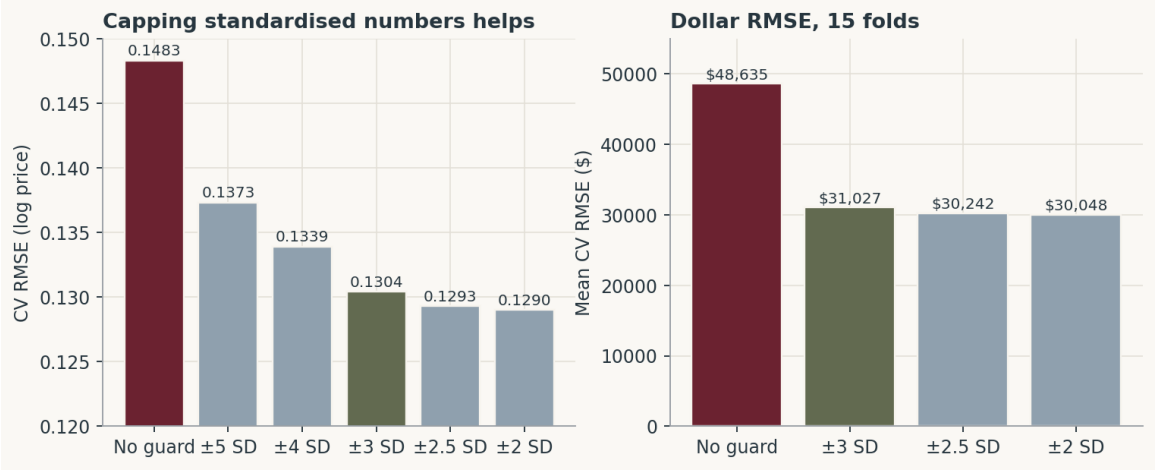


Figure 6.9. Capping standardised numbers at ±3 SD removes the extrapolation problem.

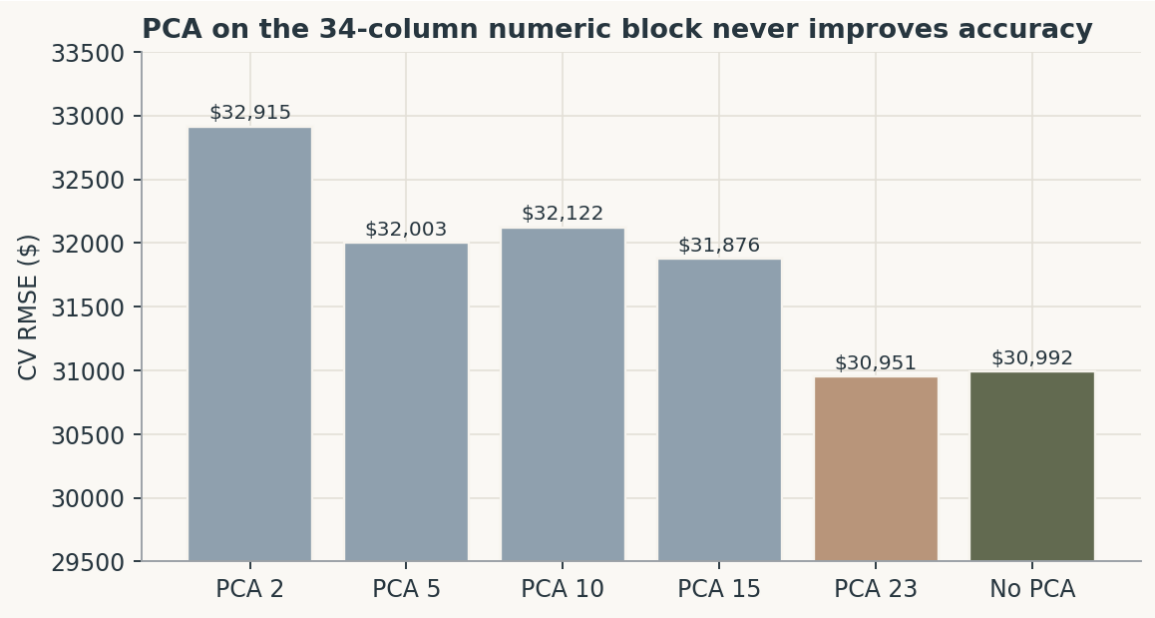


Figure 6.10. Numeric-branch PCA does not establish a meaningful improvement in cross-validated RMSE.

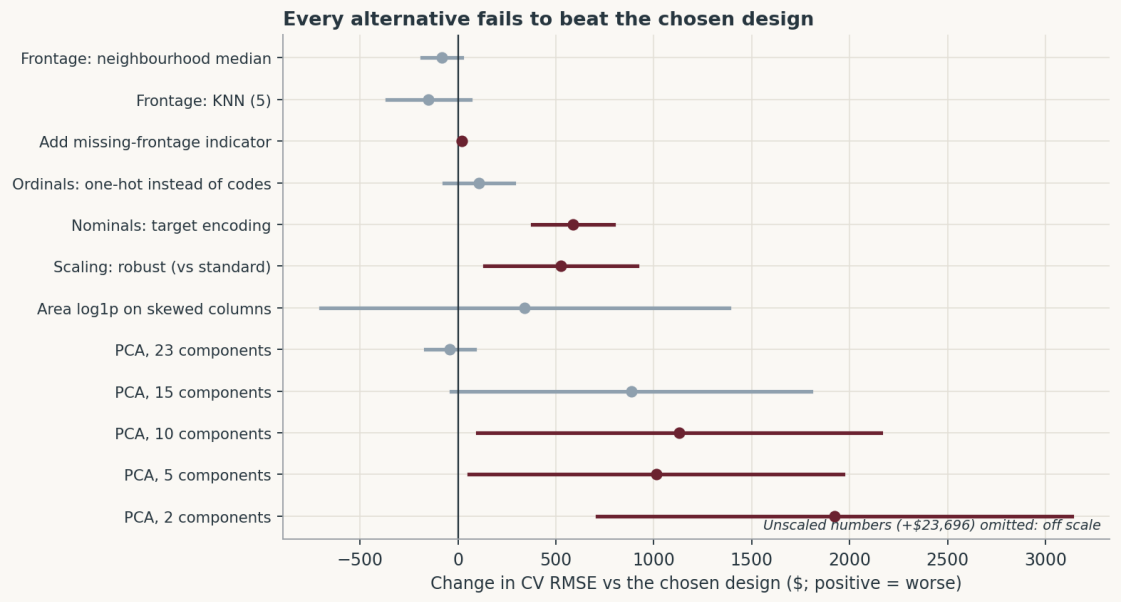


Figure 6.11. Change in cross-validated RMSE for each alternative, with 95% intervals.

ENGINEERED FEATURE GROUP	LOG RMSE	CHANGE	95% INTERVAL	ADOPTED
BASELINE (NONE)	0.1302	–	–	–
+ AGE FEATURES	0.1303	+0.00003	−0.00017 to +0.00026	no
+ TOTAL LIVING AREA	0.1295	−0.00077	−0.00094 to −0.00060	no
+ OUTDOOR AREA	0.1303	+0.00010	−0.00000 to +0.00024	no
+ TOTAL BATHROOMS	0.1303	+0.00010	+0.00004 to +0.00017	no
+ QUALITY × AREA	0.1298	−0.00042	−0.00069 to −0.00015	no
+ ALL ENGINEERED	0.1294	−0.00083	−0.00124 to −0.00039	no

No group reaches the pre-set threshold of 0.002. Three are detectable but small (about 1.2% of RMSE at best); many are linear combinations of existing columns, so a linear model gains little. The original columns were retained unchanged.

> **EDA summary:** Price is driven by quality, size and location. The data are skewed and full of structural zeros, redundancy is built into the columns, and two unusual new-build sales can derail a linear model unless extreme values are capped. These findings fixed the design: keep every column, encode by kind, scale and cap numbers, and model log price with a penalty.

7. Learning Outcomes

The project built the following skills, each tied to evidence in the notebook and this report.

LEARNING OUTCOME	WHERE IT SHOWS
Cleaning data with structural “na” values. Distinguishing “absent” from “unknown”, and imputing inside cross-validation folds so no validation row influences a fill.	Sections 2.3, 6.1–6.2
Handling ordinal and nominal columns separately. Building a metadata sheet when none is supplied, testing each assumed order against the data, and finding that five orders were unsupported.	Section 2.2, Appendix B
Why scaling, pca and fillna() must be fitted inside training folds, and how a behavioural test shows it (the fitted scaler means equal the training-fold means and differ from the all-rows means; 7 of 7 leakage checks pass).	Sections 2.3, 6.8
Diagnosing a model failure from fold-level errors. Three folds had dollar rmse above $100,000 because of two properties; a ±3 sd tail guard cut mean rmse from $48,635 to $31,027.	Sections 2.4, 6.5, 6.8
Comparing shortlisted models with nested hyperparameter tuning while recognising that earlier preprocessing and shortlisting decisions used the same training dataset.	Section 8.1
Reporting honestly. Confidence intervals, a disclosed earlier look at the held-out rows, post-hoc subgroup limits, and a refusal to call a model change “validated” because it improved a number.	Sections 1.4, 8.2, 8.5
Judging prediction intervals by coverage, including where they fail: 67% in the top price quarter against an 80% target.	Section 8.4
Reading association, not causation. Coefficients, permutation importance and partial dependence describe what the model learned, not what changing a property would do.	Section 8.3
Reproducibility. Named seeds, a file fingerprint, a logged test vault, an importable pipeline module and a model reloaded in a fresh process with zero prediction difference.	Sections 3.1, 4

8. Conclusion

This section reports the modelling results, the final evaluation and the limits of what they show, and then closes the report. Figures in 8.1–8.4 combine the notebook’s printed results with plots redrawn from the frozen model where noted.

8.1 Model development and selection

Ten algorithms were scored on the same 15 folds (5 × 3 repeated K-fold, seed 2026), each with 934 or 935 training rows and 233 or 234 validation rows. Preprocessing sits inside every estimator and the log-price target is converted back to dollars before scoring. Three baselines set the floor: predicting the mean gives RMSE $81,293 ($R^2$ −0.006), the median $83,131 ($R^2$ −0.051), and a five-feature OLS (GrLivArea, OverallQual, YearBuilt, PropertySize, BasementCars) gives RMSE $40,774, MAE $26,713 and R^2 0.745. Seven of ten candidates beat that OLS.

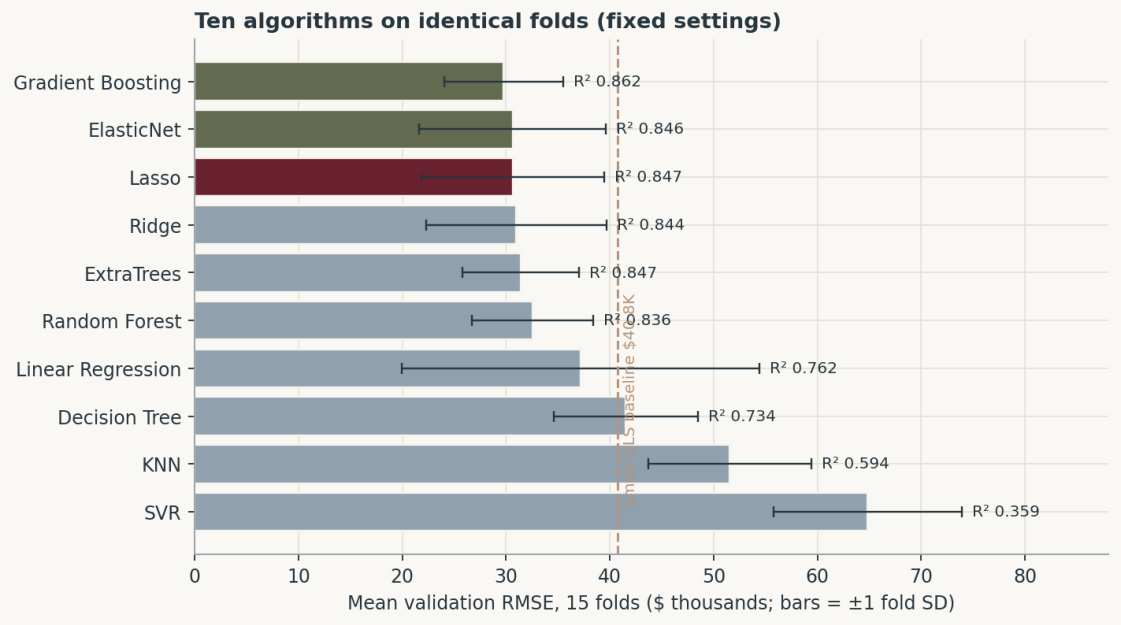


Figure 8.1. Mean validation RMSE of ten algorithms on identical folds (fixed settings).

MODEL	RMSE ($)	FOLD SD ($)	MAE ($)	R^2	LOG RMSE
GRADIENT BOOSTING	29,779	5,705	16,705	0.862	0.132
ELASTICNET	30,628	8,971	15,651	0.846	0.129
LASSO	30,661	8,816	15,691	0.847	0.129
RIDGE	30,992	8,665	16,200	0.844	0.130
EXTRATREES	31,420	5,611	17,939	0.847	0.145
RANDOM FOREST	32,545	5,841	18,429	0.836	0.147
LINEAR REGRESSION	37,143	17,229	17,780	0.762	0.147
DECISION TREE (DEPTH 6)	41,544	6,968	26,830	0.734	0.201
KNN	51,561	7,872	31,489	0.594	0.237
SVR	64,839	9,085	40,083	0.359	0.288

The metrics favour different candidates: Gradient Boosting has the lowest initial mean RMSE, while penalised linear models perform better on MAE and log RMSE. Unregularised linear regression has substantial fold variability (RMSE SD $17,229). The shallow decision tree is worse than the five-feature OLS baseline, with RMSE $41,544 versus $40,774. KNN and SVR perform poorly under their tested settings. All candidates use the shared Section 9 preprocessing, with encoded dimensionality potentially varying across folds. These fixed-setting comparisons are exploratory rather than definitive rankings of the algorithm families.

Nested tuning and the choice of Lasso

The three shortlisted models are tuned using three inner folds within each of five outer folds. Nested mean RMSE is $30,377 for Lasso, $30,503 for ElasticNet and $30,649 for Gradient Boosting. Exploratory paired RMSE intervals include zero, so the comparison does not establish a clear winner or demonstrate equivalence. Lasso has lower MAE than Gradient Boosting in all five outer folds, with a mean difference of −$1,358 and interval −$2,435 to −$281. The RMSE difference is −$272, with interval −$7,064 to +$6,519. Lasso is selected for the documented balance of error, simplicity and interpretability. The initial and nested comparisons use different evaluation designs, so their score difference alone does not prove optimism. Earlier preprocessing and shortlisting decisions also used this training dataset, limiting the independence of the nested assessment.

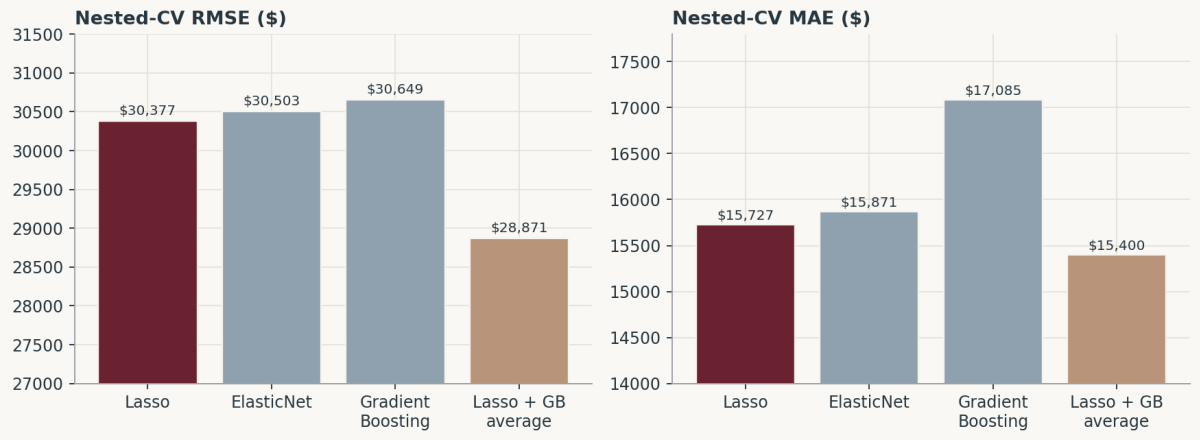


Figure 8.2. Nested cross-validation RMSE and MAE for the shortlisted models and the Lasso + Gradient Boosting average.

Averaging Lasso and Gradient Boosting scored lower still (nested RMSE $28,871, MAE $15,400, R^2 0.869) but did not meet the adoption rule: it beats Lasso on dollar RMSE in only 3 of 5 folds with an interval that includes zero (−$1,506, interval −$4,917 to +$1,905). It is recorded as a possible future improvement. Two further checks changed nothing: Duan smearing (Section 6.3) and removing SaleType and SaleCondn (RMSE $30,940 against $30,992). Those two columns describe the sale rather than the property and would not be known when valuing a house in advance, but they were retained as the brief requires.

8.2 Final held-out evaluation

The estimator was frozen (Lasso, alpha 0.00131 from a training-only search, configuration SHA-256 863b2ba5…) before the held-out rows were opened, and the held-out rows were opened once for the final evaluation. Nothing was changed afterwards.

METRIC	HELD-OUT RESULT	95% BOOTSTRAP INTERVAL	CROSS-VALIDATION (TRAINING ROWS)
R^2	0.930	0.912 – 0.948	0.847 (fold SD 0.082)
MAE	$12,828	$11,399 – $14,359	$15,817
RMSE	$18,512	$15,508 – $22,086	$30,701
LOG RMSE	0.117	0.094 – 0.139	0.129

A typical error is about $12,800, roughly 8% of a typical sale price. The held-out result is better than cross-validation by 0.084 in R^2 (about one fold SD) and $12,189 in RMSE (1.4 fold SDs). The most likely reason is that all four training properties above 4,000 sq ft (IDs 524, 692, 1183 and 1299) are in the training set, so no held-out row needs that kind of extrapolation; the largest held-out error is $131,772. This is a reading of the evidence, not a tested result. The bootstrap intervals describe sampling variation in these 292 rows, not the uncertainty of a new market.

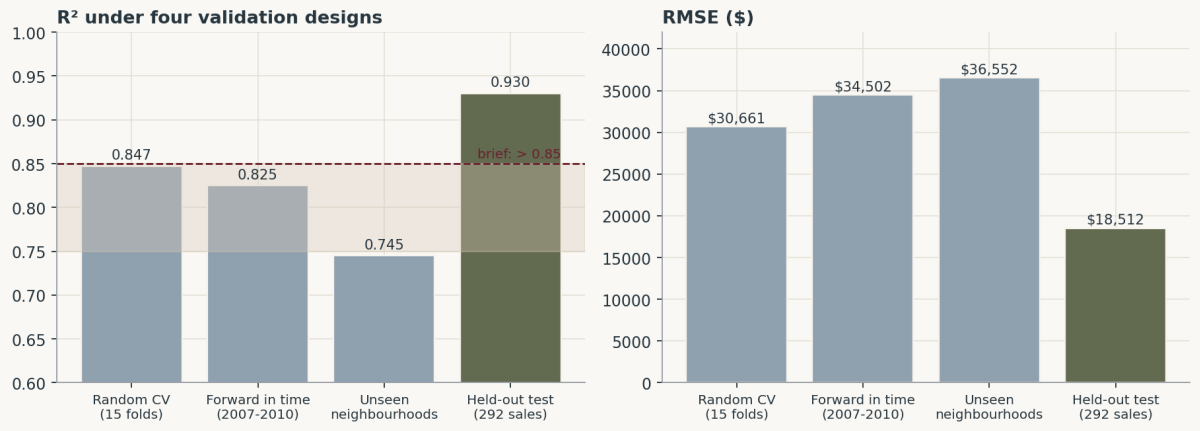


Figure 8.3. R^2 and RMSE under four validation designs; the held-out result is the most favourable.

Two stricter designs, run on training rows only with the Section 10 Lasso, show less flattering but more honest numbers. Holding out whole neighborhoods gives R^2 0.745, MAE $18,484 and RMSE $36,552, with one fold collapsing to R^2 0.312 (RMSE $61,544). Forward-in-time validation (fit on earlier sale years, predict the next) gives pooled R^2 0.825, MAE $17,239 and RMSE $34,502, with the weakest year 2007 (RMSE $41,991). Predicting a neighborhood or period the model has not seen is clearly harder.

> **So what:** Quote R^2 0.847 and RMSE of about $30,700 as the expected accuracy for sales like those in the training data. The held-out 0.930 is supportive but comes from rows that were opened once before in an earlier model version, and from a test set that happens to lack the hardest properties.

8.3 What drives price

Original-feature permutation importance refits the frozen model specification on three training-only folds and permutes one source variable at a time in each validation fold, with two permutations per feature. Preprocessing then transforms the permuted input, grouping its encoded representation. Mean RMSE increases are $14,871 for GrLivArea, $8,978 for OverallQual, $2,356 for BsmntSqFtage, $2,346 for YearBuilt, $1,780 for OverallCond, $1,704 for BasementCars and $1,594 for Neighborhood. GrLivArea's increase ranges from $3,511 to $21,186 across evaluations, showing substantial variability. Correlated predictors can share or mask predictive information, so Neighborhood's lower conditional ranking is not inconsistent with its strong univariate association. These results describe predictive reliance rather than causal effects.

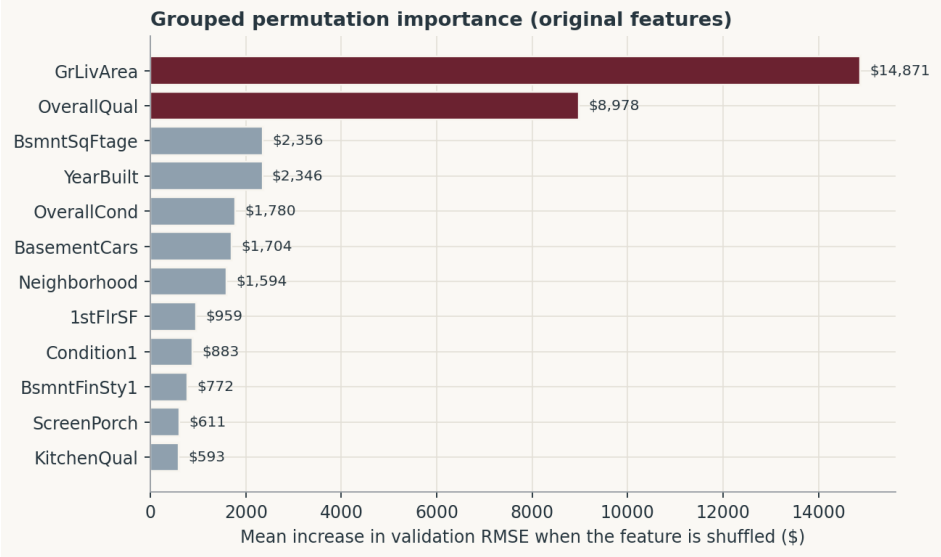


Figure 8.4. Grouped permutation importance of the twelve most important original features.

In the final fit, 71 of 200 encoded terms are non-zero. For an unclipped one-standard-deviation increase within the main standardised numeric branch, the living-area and overall-quality coefficients correspond to approximate prediction changes of 14.6% and 8.7%, holding other model inputs fixed. Clipping limits this interpretation near the transformation boundaries. The nominal encoder retains all resulting categories rather than dropping a reference level, so individual category coefficients must not be interpreted as differences from an omitted category. A comparison between categories A and B uses 100 × [exp(βA − βB) − 1]%, conditional on the fitted specification. Across 30 bootstrap refits, 20 terms are selected in every refit and 42 in at least 80%. These frequencies are approximate stability diagnostics, and correlated predictors can substitute for one another.

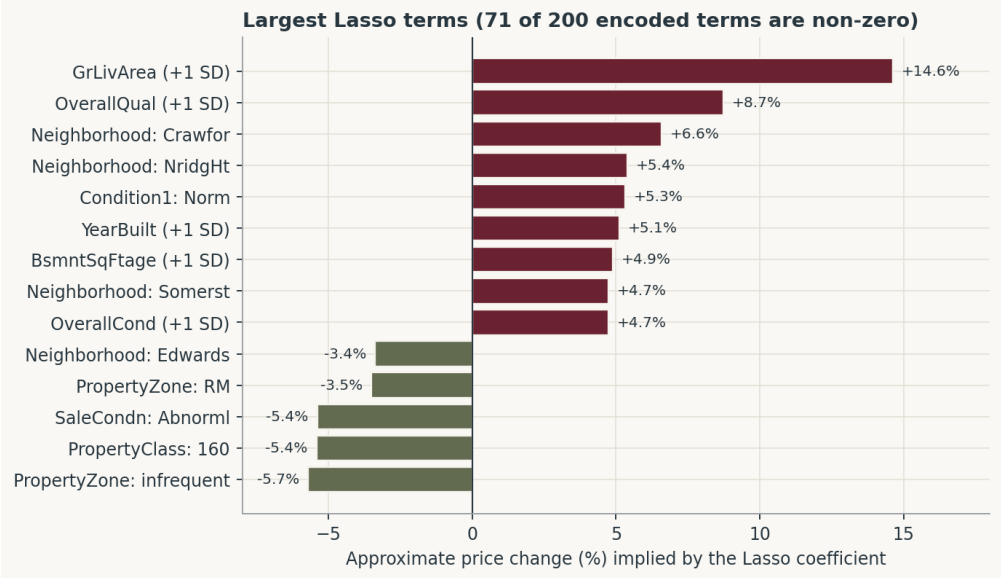


Figure 8.5. Exponentiated Lasso coefficients. Standardised numeric coefficients describe changes within the unclipped range. Individual category coefficients are not contrasts against an omitted category.

8.4 Where the model is weak

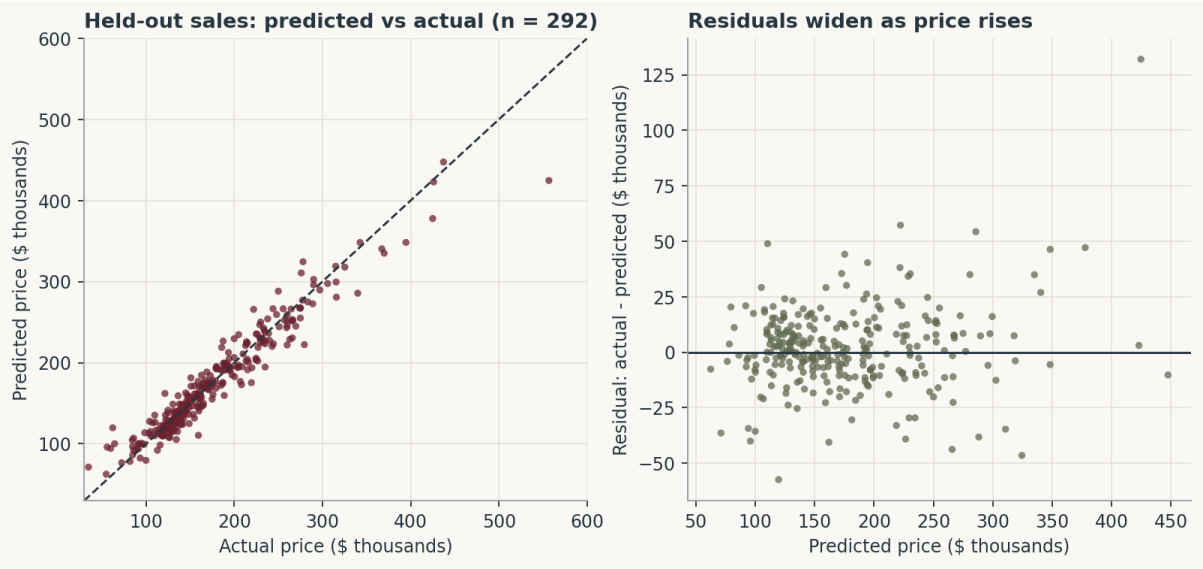


Figure 8.6. Held-out predicted against actual price, and residuals (redrawn by re-running the frozen pipeline on the same split; headline metrics are quoted from the notebook's run).

Error grows with price (Breusch–Pagan LM = 25.3, p = 5e−07 on the held-out residuals). The cheapest three quarters of sales have MAE between $9,709 and $11,160, while the top quarter has MAE $19,326 (RMSE $27,411) and a mean residual of +$10,013: the most expensive homes are under-predicted. Eleven of the 15 largest held-out errors are in the top quarter and four are Partial new-construction sales. The largest is a StoneBr Partial sale at $556,581 predicted as $424,809 (residual +$131,772).

SALE CONDITION	N	MAE ($)	RMSE ($)	MEAN RESIDUAL ($)
PARTIAL (NEW CONSTRUCTION)	30	19,743	32,347	+12,875
ABNORMAL	23	12,917	18,031	−49
NORMAL	233	11,734	15,792	+1,370

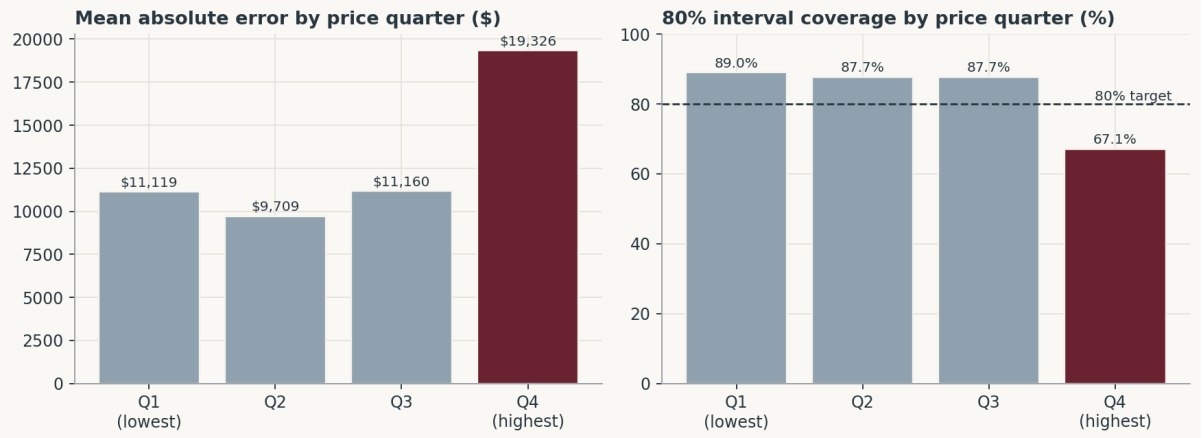


Figure 8.7. Held-out MAE by price quarter (left) and coverage of the 80% prediction intervals (right).

The split-conformal procedure fits a separate clone of the selected Lasso on 934 proper-training rows and calibrates absolute errors on the remaining 234 training rows. Intervals are centred on predictions from that separately fitted estimator, rather than the full-training estimator used for the headline point metrics. The radius is $20,597 and the width is $41,194. Observed test coverage is 82.9% (Wilson 95% interval

78.1%–86.8%), with coverage of 89.0%, 87.7%, 87.7% and 67.1% across ascending price quarters. Partial-sale coverage is 70.0%. These are empirical coverage results; the independence limitation in Section 8.5 qualifies a formal coverage guarantee. Separately, the headline point model's neighbourhood MAE is highest in StoneBr ($36,962, n = 5), ClearCr ($26,360, n = 7) and Crawfor ($19,647, n = 12), and lowest in NAmes ($7,594, n = 45). Small groups make these rankings indicative only (Appendix C).

8.5 Limitations

- **One market and period.** Residential sales in Ames, Iowa, 2006–2010. Accuracy elsewhere, in later markets or under different economic conditions has not been shown.
- **Renamed adaptation and own split.** The file is a renamed version of the Ames data and only one CSV was supplied, so the split is our own and metrics may not be comparable with published benchmarks or other splits.
- **Small held-out sample.** 292 sales; subgroup estimates (especially neighborhoods with a handful of sales) are imprecise.
- **Test-set exposure.** An earlier Gradient Boosting version scored R^2 0.912 on the same held-out rows. The held-out rows are not historically untouched; cross-validation is the primary evidence, and all residual and subgroup analyses are post-hoc.
- **Post-hoc methodological refinement.** The ±3 SD tail guard was introduced after training-only diagnostics showed extrapolation, and the choice among guards was amended after seeing the **results. It is a documented amendment, not a fully pre-specified decision.**
- **Repeated analytical selection.** Preprocessing choices, model shortlisting and methodological amendments were informed by the same training dataset. Nested tuning separates its inner searches from outer scoring, but does not independently validate every preceding analytical decision. Reported training-CV results may therefore retain selection optimism.
- **Conformal calibration independence.** The Lasso hyperparameters were selected using all training rows before the proper-training/calibration split. Calibration observations were therefore not fully independent of model selection. A stricter split-conformal workflow would reserve calibration rows before preprocessing and hyperparameter selection, then fit the selected model using proper-training rows only.
- **Predictors known only after sale.** SaleType and SaleCondn describe the transaction and may be unavailable when pricing a house before sale.
- **Extrapolation and rare properties.** Only four training properties exceed 4,000 sq ft. The tail guard limits extreme transformed values but does not make large-property predictions reliable.
- **Uneven accuracy.** Errors are larger for expensive properties and Partial sales; prediction intervals under-cover these groups (about 67% and 70% against an 80% target).
- **Unseen neighborhoods and later years are harder** (R^2 0.745 and 0.825 in the stricter checks above).
- **Association, not causation.** Coefficients, permutation importance and partial dependence describe the fitted model.
- **Absence versus missing.** The "None" treatment must stay consistent when the model scores new properties. Alley and fence readings of "NA" are assumptions without a companion column.
- **Deployment not demonstrated.** The model reloads in a fresh process with zero prediction difference, but monitoring, drift detection, retraining and external validation were not done.

8.6 Overall conclusion

This project developed and evaluated regression models for predicting residential property prices in Ames, Iowa, using all 79 original predictors and a documented, leakage-controlled preprocessing pipeline. A Lasso regression on log price, selected by nested cross-validation, reaches cross-validated R^2 of 0.847 ± 0.082 (RMSE about $30,700) and, on 292 held-out sales, R^2 0.930 (95% interval 0.912 to 0.948), MAE $12,828 and RMSE $18,512, meeting the brief's expectation of 0.75–0.85 in cross-validation and exceeding 0.85 on the held-out rows. Living area, overall quality, construction year, basement area, condition, garage capacity and neighborhood were the most influential predictive characteristics.

The strongest evidence comes from training-only cross-validation. The held-out score, subgroup diagnostics and stricter validations clarify both the model's potential and its limits: it is least reliable for expensive homes, new-construction sales, unseen neighborhoods and properties unlike those it was trained on.

Expected outcomes from the brief

EXPECTED OUTCOME	DELIVERED
A MODEL THAT PREDICTS PROPERTY PRICES ACCURATELY IN A SPECIFIC LOCATION	Lasso on log price for Ames, Iowa; CV R^2 0.847, held-out R^2 0.930 (Sections 8.1–8.2).
INSIGHTS INTO THE SIGNIFICANT FEATURES	Living area and overall quality dominate; neighborhood tiers; structural absence carries information (Sections 6.6, 8.3).
A COMPREHENSIVE REPORT ON METHODOLOGY, RESULTS AND INSIGHTS	This report, with the Jupyter notebook as the full record.

Recommendations and next steps

- Quote cross-validated accuracy first, and report the held-out result with its exposure history.
- Treat the equal-weight Lasso + Gradient Boosting average as the first improvement to test (nested RMSE $28,871) under a fresh, untouched test set.
- Add price-dependent prediction intervals, since a fixed width under-covers expensive homes and Partial sales.
- Before any use beyond Ames or for pre-sale valuation, validate on another market and remove or justify SaleType and SaleCondn.

DELIVERABLE	STATUS
JUPYTER NOTEBOOK	Complete, 12 sections; submitted with this report
PROJECT SUMMARY REPORT (WORD FILE)	This document, in the nine-section format of the brief

9. Citations – Books and Websites Used for Research

Dataset and background

[1] De Cock, D. (2011). Ames, Iowa: Alternative to the Boston housing data as an end of semester regression project. *Journal of Statistics Education, 19*(3). https://doi.org/10.1080/10691898.2011.11889627

Books

[2] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7

[3] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning: With Applications in R* (2nd ed.). Springer. https://doi.org/10.1007/978-1-0716-1418-1

[4] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.

[5] Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall/CRC.

Papers

[6] Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B, 58*(1), 267–288.

[7] Duan, N. (1983). Smearing estimate: A nonparametric retransformation method. *Journal of the American Statistical Association, 78*(383), 605–610. https://doi.org/10.1080/01621459.1983.10478017

[8] Lei, J., G'Sell, M., Rinaldo, A., Tibshirani, R. J., & Wasserman, L. (2018). Distribution-free predictive inference for regression. *Journal of the American Statistical Association, 113*(523), 1094–1111. https://doi.org/10.1080/01621459.2017.1307116

[9] Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B, 57*(1), 289–300.

Software and websites

[10] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

[11] Scikit-learn developers. (n.d.). *Scikit-learn user guide*. https://scikit-learn.org/stable/user_guide.html

[12] pandas development team. (n.d.). *pandas documentation*. https://pandas.pydata.org/docs/

[13] McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.

[14] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95.

[15] Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. *Proceedings of the 9th Python in Science Conference*, 92–96.

Appendix A : Brief-to-CSV name map and corrected meanings

Seven positions differ between the brief's field list and the CSV. Position and content agree, so the mapping is safe. The CSV names are used throughout.

POSITION	BRIEF NAME	CSV NAME	NOTE
10	LotOrientation	Orientation	Lot configuration
47	BsmtBath1	BsmtFullBath	Basement full bathrooms
48	BsmtBath2	BsmtHalfBath	Basement half bathrooms
52	Kitchen	KitchenUpLev	Kitchens above grade
62	SquareFootage	BasementSqFootage	Garage area (the name says basement)
64	BasementSqFootage	BasementCond	Garage condition
77	YrSold	SaleYr	Year sold

The seven columns from BasementType to BasementCond are described by the brief as garage (location, year built, finish, car capacity, area, quality, condition) and hold garage labels such as Attchd, Detchd and CarPort. Roof1Material and Roof2Material are described as exterior covering and share no label with RoofMatl, the real roof material. The BsmntFin… descriptions fit their content, so no swap was found, although BsmntFinSty1 holds an area despite "Sty" in its name.

Appendix B : Ordinal scales (low to high) and order checks

COLUMN	DECLARED ORDER, LOW → HIGH	P WITH LOG PRICE	SUPPORTED?
EXTERQUAL	Po < Fa < TA < Gd < Ex	+0.69	yes
EXTERCOND	Po < Fa < TA < Gd < Ex	+0.03	no: too weak
HEATINGEFFICIENCY	Po < Fa < TA < Gd < Ex	+0.50	yes
KITCHENQUAL	Po < Fa < TA < Gd < Ex	+0.69	yes
BSMNTFINISH	None < Po < Fa < TA < Gd < Ex	+0.68	yes
BSMNTMAINTENANCE	None < Po < Fa < TA < Gd < Ex	+0.27	yes
BSMNTVISIBILITY	None < No < Mn < Av < Gd	+0.35	yes
BSMNTFINRAT1	None < Unf < LwQ < Rec < BLQ < ALQ < GLQ	+0.38	no: mixed order
BSMNTFINQUAL1	None < Unf < LwQ < Rec < BLQ < ALQ < GLQ	+0.05	no: mixed order
FUNCTIONAL	Sal < Sev < Maj2 < Maj1 < Mod < Min2 < Min1 < Typ	+0.13	untestable
QUALFIREPLACE	None < Po < Fa < TA < Gd < Ex	+0.54	yes
BASEMENTFINISH (GARAGE)	None < Unf < RFn < Fin	+0.64	yes
BASEMENTQUAL (GARAGE)	None < Po < Fa < TA < Gd < Ex	+0.36	yes
BASEMENTCOND (GARAGE)	None < Po < Fa < TA < Gd < Ex	+0.35	yes
PAVEDDRIVE	N < P < Y	+0.29	yes
POOLQC	None < Fa < TA < Gd < Ex	+0.07	untestable
BOUNDARYFEATURES	None < MnWw < GdWo < MnPrv < GdPrv	−0.19	no: mixed order
PROPERTYSHAPE	IR3 < IR2 < IR1 < Reg	−0.32	yes (falling)
GRADE	Gtl < Mod < Sev	+0.04	no: too weak

Binary columns: Street (Grvl = 0, Pave = 1), CentralAir (N = 0, Y = 1), Amenities (NoSeWa = 0, AllPub = 1). "None" (feature absent) is placed below the lowest grade; this is an assumption. The order of Rec and BLQ in the basement finish scales, and the fence order, are also assumptions.

Appendix C : Held-out error detail (post-hoc)

Table C.1. Error by neighborhood (neighborhoods with at least 5 held-out sales; small groups are indicative only).

NEIGHBORHOOD	N	MAE ($)	RMSE ($)	MEAN RESIDUAL ($)
STONEBR	5	36,962	60,234	+32,680
CLEARCR	7	26,360	29,389	+14,352
CRAWFOR	12	19,647	23,817	+10,218
SAWYER	9	18,990	25,452	−5,439
SWISU	7	18,913	23,602	+2,368
TIMBER	8	18,091	21,306	+13,842
IDOTRR	6	17,504	22,937	−15,635
NORIDGE	9	17,037	20,753	+834
SOMERST	21	15,566	20,104	+10,903
BRKSIDE	10	14,609	17,367	+13,385
NWAMES	15	12,992	18,019	−3,588
NRIDGHT	13	12,342	18,374	+1,668
SAWYERW	13	12,054	13,971	−5,481
EDWARDS	21	11,684	14,536	+1,924
COLLGCR	35	10,974	16,448	+3,619
OLDTOWN	26	9,929	12,658	−324
BLMNGTN	6	9,275	9,880	−5,679
MITCHEL	10	8,327	11,940	−3,854
NAMES	45	7,594	9,907	+566

Table C.2. The fifteen largest held-out errors.

ACTUAL ($)	PREDICTED ($)	RESIDUAL ($)	ERROR %	NEIGHBORHOOD	SALE CONDITION	SALE TYPE
556,581	424,809	+131,772	23.7	StoneBr	Partial	New
62,383	119,771	−57,388	92.0	Sawyer	Normal	WD
279,500	222,393	+57,107	20.4	CollgCr	Partial	New
340,000	285,875	+54,125	15.9	Somerst	Normal	WD
159,434	109,734	+49,700	31.2	SWISU	Abnorml	WD
424,870	377,941	+46,929	11.0	CollgCr	Partial	New
278,000	324,458	−46,458	16.7	NWAmes	Normal	WD
394,617	348,455	+46,162	11.7	NridgHt	Partial	New
219,500	175,458	+44,042	20.1	Crawfor	Normal	CWD
222,000	265,923	−43,923	19.8	Crawfor	Normal	WD
121,500	162,177	−40,677	33.5	Sawyer	Normal	WD
235,000	194,759	+40,241	17.1	ClearCr	Normal	WD
55,993	95,345	−39,352	70.3	IDOTRR	Alloca	WD
250,000	288,378	−38,378	15.4	NoRidge	Normal	WD
260,000	221,996	+38,004	14.6	ClearCr	Normal	WD

Eleven of the fifteen are in the top price quarter and four are Partial new-construction sales.

Appendix D : Plain-language notes on the statistics

TERM	MEANING IN THIS REPORT
LASSO	A linear model with a penalty that shrinks coefficients and sets some exactly to zero. Zeroed terms are model behaviour, not dropped columns.
LOG TARGET	The model predicts log(price) and the result is converted back with exp(). It makes errors proportional to price rather than constant in dollars.

R^2, MAE, RMSE	R^2 = 1 is a perfect fit and 0 is no better than the mean. MAE is the typical error in dollars; RMSE punishes large errors more.
CROSS-VALIDATION (5 × 3)	Training rows are split into five folds, each used once for validation, repeated three times (15 folds). Every fitted value is learned from the other folds only.
NESTED CROSS-VALIDATION	Tuning happens inside each training fold so the outer score is not flattered by choosing settings on the same data.
HELD-OUT TEST	292 rows set aside before analysis and scored once with the frozen model.
BOOTSTRAP INTERVAL	Resampling the test rows many times to see how much a score could vary by chance.
SPEARMAN P	How consistently two measures rise together (−1 to +1), using ranks, so it is less sensitive to extreme values.
RANK E^2 (EPSILON-SQUARED)	Share of the rank variation in price that a categorical predictor explains, from a Kruskal–Wallis test.
CRAMÉR'S V	Strength of association between two categorical variables (0 to 1).
VIF	How much a predictor's coefficient variance is inflated by overlap with other predictors; above 10 is high.
PCA	Combines correlated numeric columns into a few components. Here it compressed the numbers but never improved prediction.
TAIL GUARD (±3 SD)	Standardised numbers beyond three standard deviations are capped so one extreme property cannot push a linear prediction far off.
ADVERSARIAL VALIDATION	Asking a model to tell training rows from test rows using predictors only; an AUC near chance means the two parts look alike.
SPLIT CONFORMAL INTERVAL	A prediction range of fixed width, calibrated on training rows, designed to cover the true price about 80% of the time.
PERMUTATION IMPORTANCE	How much validation error rises when one original feature is shuffled; larger means the model leans on it more.